Project - Phase 1

Mahan Zamani(402101877) - Sina Hashemi(402102668)

1 Project Description and Theoretical Questions

1.1 Introduction to Singal Noise Cancelling

1. Conduct research on the introduced noise models and describe them in your own words. Provide an explanation of these models in the context of discrete-time signals, and formulate them within the discrete-time domain.

    - **Gaussian noise:** A Gaussian (normal) random noise has each sample drawn from a gaussian distribution. It is fully characterized by its mean and variance(μ and σ).In discrete-time, each sample is an independent random variable drawn from a Gaussian distribution, $x[n] \sim \mathcal{N}(0, \sigma^2)$.

    - **Uniform noise**: Uniform noise has samples equally likely over an interval $[-A : A]$. This means all values within the range [-A, A] are equally likely to occur. The discrete-time version has each sample $x[n]$ uniformly distributed between $-A$ and $+A$, with mean = 0 and variance = $\frac{A^2}{3}$.

    - **White noise**: White noise is a sound that contains all frequencies within the range of human hearing. In practice, one generates white noise by drawing samples from any zero-mean distribution (commonly Gaussian) that are uncorrelated, means that $x[n] \sim \mathcal{N}(0, \sigma^2) uncorrelated$

    - **Impulse noise**: Impulse noise consists of short-duration, high-amplitude spikes that occur randomly. It's common in digital communication systems and can result from switching transients, lightning, or other sudden disturbances. Ih discrete-time, $x[n] = \delta[n]$ or in more genreal form $x[n] = \delta[n - n_0]$.

    - **Power line noise**: This noise stems from AC power sources (50 Hz for Iran). In discrete-time, it is a sampled sinusoid at the power line frequency. $x[n] = A\sin(2\pi\frac{f}{F_s}n + \phi)$ with $f = 50Hz$ and F_s the sampling rate.

---

2. Research the introduced noise-canceling methods and explain them in your own words. Make sure to formulate these methods using mathematical equations. Additionally, discuss the advantages and disadvantages of each method. (Note: You do not need to explain the Adaptive Filtering method in this question.)

- **FIR filtering**: A Finite Impulse Response (FIR) filter uses a finite number of input samples to compute its output. It operates via convolution between the input signal and a set of filter coefficients.

$$y[n] = \sum_{k=0}^{N-1} h[k]x[n-k]$$

where $h[n]$ are the filter coefficients and N is the filter length. The transfer function in Z-domain is:

$$H(z) = \sum_{k=0}^{N-1} h[k]z^{-k}$$

Advantages:

```
 - Guaranteed stability: All poles at the origin.
 - Linear Phase: Preserves signal timing relationships.
 - No feedback, so finite precision effects are minimal.
 - Design simplicity.
```

Disadvantages:

```
 - Long filter lengths: Sharp cutoffs require large N,
increasing latency & compute.
 - Fixed coefficients: Cannot adapt to time-varying noise.
```

- **IIR filtering**: An Infinite Impulse Response (IIR) filter uses feedback, allowing its output to depend on both input and past output values. This recursive structure enables efficient frequency-selective filtering with fewer coefficients than FIR filters.

$$y[n] = \sum_{k=0}^{N} b_k x[n-k] - \sum_{k=1}^{M} a_k y[n-k]$$

The transfer function in Z-domain is:

$$H(z) = \frac{\sum_{k=0}^{N} b_k z^{-k}}{1 + \sum_{k=1}^{M} a_k z^{-k}} = \frac{B(z)}{A(z)}$$

Advantages:

```
 - Efficiency: Achieves sharp roll-offs (e.g.,
lowpass/highpass) with fewer taps than FIR.
 - Flexibility: Can model resonant or recursive systems.
```

Disadvantages:

```
 - Nonlinear Phase: Feedback introduces phase distortion.
 - Potential Instability: Poorly chosen a[m] can place poles
outside the unit circle.
 - Complex Design: Requires iterative methods for coefficient
tuning.
```

1.2 Eigen Signals

1. Explain the concept of eigen signals in your own words. How are eigen signals used in signal processing? What are the advantages of using eigen signals?

Eigen signals (also called principal components) are simply the eigenvectors of a signal's covariance matrix. These signals are optimized to represent the underlying structure of a dataset with minimal redundancy, making them ideal for tasks like compression, feature extraction, and noise removal.

$$R = \mathcal{E}[xx^T], \qquad Ru_k = \lambda_k u_k, \qquad k = 1, \dots N$$

Each u_k is an eigen signal of x. Thus,

$$x = \sum_{k=1}^{N} a_k u_k$$

- Applications in Signal Processing

1. Transform Domain Processing: Because the eigen-signals diagonalize the covariance, the transform coefficients are uncorrelated. So using eigen signals we can transform original signals into some coefficients and analyse them easier. For example Fourier Transform uses complex exponentials $e^{j2\pi ft}$) as eigen signals.

2. Image Compression: In JPEG compression, the DCT decomposes image blocks into frequency components and Most energy concentrates in low-frequency coefficients, allowing high compression ratios. Also by Keeping only the first K components with largest coefficients we can compress image.

3. Noise Cancellation: If noise is roughly white, it spreads energy evenly across all eigen-signals, whereas the signal of interest concentrates in the top few. Projecting onto the top K eigen-signals suppresses noise subspace.

- Advantages of Eigen Signals

1. Compact Representation: Signals are represented with fewer coefficients, reducing storage/bandwidth (critical for compression).

2. Orthogonality: Mutual orthogonality eliminates redundancy between components, improving efficiency.

3. Energy Compaction: Dominant eigen signals concentrate most signal energy, enabling lossy compression with minimal distortion.

4. Noise Robustness: Noise typically correlates weakly with eigen signals, making them effective for denoising.

1.3 Adaptive Filtering

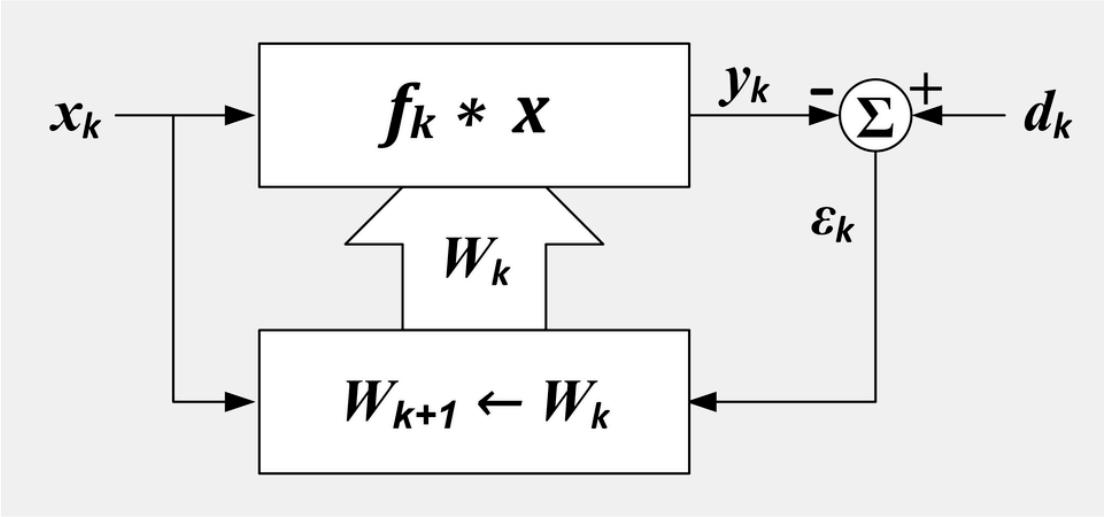
1. Provide the mathematical formulation of the adaptive filtering algorithm. Clearly explain each step of the algorithm in your own words, ensuring specificity and clarity in every sentence. Using pictures and diagrams is strongly recommended for better understanding. Additionally, explain how eigen signals can be utilized in adaptive filtering.

An adaptive filter dynamically adjusts its filter coefficients to minimize the error between a desired signal d[n] and the filter's output y[n], based on an input signal x[n].

$$\text{Filter Output:} \qquad y(n) = \sum_{k=0}^{N-1} w_k(n) * x(n-k) = w^T(n) * x(n)$$

$$\text{Error Signal:} \qquad e(n) = d(n) - y(n)$$

$$\text{Weight Update (LMS):} \qquad w(n+1) = w(n) + \mu * e(n) * x(n)$$



Input Signals:

- x_k: Input signal vector at time k (contains current and past input samples)
- d_k: Desired signal (reference/target signal we want to match)

Filter Components:

- f_k: Adaptive filter impulse response at time k
- W_k: Filter weight/coefficient vector at time k
- W_{k+1}: Updated filter weights for the next iteration

Output Signals:

- y_k: Filter output signal at time k
- ϵ_k: Error signal = d_k - y_k

How the System Works:

1. Initialization: Set initial filter weights w(0) to small random values or zeros. Choose appropriate step size μ.

2. Input Processing: At each time step n, receive input sample x(n) and form input vector x(n) containing current and previous N-1 samples.
3. Filter Output Calculation: Compute filter output y(n) by taking dot product of weight vector w(n) and input vector x(n).
4. Error Computation: Calculate error signal e(n) as difference between desired signal d(n) and filter output y(n).
5. Weight Update: Adjust filter weights using gradient descent rule, moving in direction that minimizes mean square error(or any other loss functions).
6. Iteration: Repeat steps 2-5 for each new input sample, allowing filter to continuously adapt to changing signal characteristics.

Eigen can improve adaptive filtering in multiple ways:

- Faster Convergence: If you transform x[n] into its eigenvector basis, the components become uncorrelated so LMS converges faster.
- Subspace Tracking: You can track and cancel noise more effectively by focusing only on the signal subspace (top eigenvectors), ignoring components dominated by noise.

---

2. In adaptive filtering, selecting an appropriate loss function is essential for updating the filter coefficients. Describe the various loss functions that can be employed in adaptive filtering, with a particular emphasis on the LMS (Least Mean Squares) loss function.

- Least Mean Squares (LMS): With mathematical Formulation $J[n] = \frac{1}{2}e^2[n]$ with gradient $\nabla J[n] = -e(n)x(n)$, Thus we have $w(n+1) = w(n) - \mu\nabla J(n) = w(n) + \mu e(n)x(n)$. This function has Low complexity and it's suitable for real-time. but it's sensitive to μ and input power.
- Mean Absolute Error (MAE): With loss function $J[n] = |e[n]| = |d[n] - y[n]|$.
- Least Mean Fourth (LMF): Using $J = E[e^4(n)]$.
- Huber Loss: Hybrid model of MSE and MAE,

$$J[n] = \begin{cases} \frac{1}{2}e^2[n] & \text{if } |e[n]| \leq \delta \\ \delta(|e[n]| - \frac{1}{2}\delta) & \text{if } |e[n]| > \delta \end{cases}$$

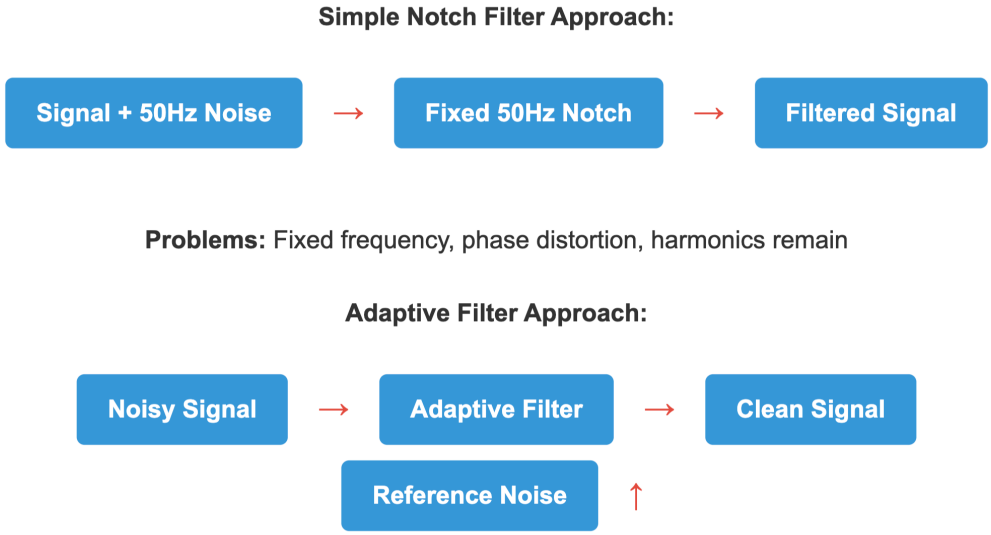
---

3. What is the primary advantage of adaptive filtering compared to simple filtering? Specifically, explain how adaptive filtering can be applied to remove power line noise from a signal and why it is more beneficial than using simple notch filtering.

The Primary Advantage of Adaptive Filtering is Dynamic Adaptation, This filter adjusts to changing noise statistics (e.g., frequency drift, amplitude changes), unlike fixed-coefficient filters.

Notch Filter fails if the noise drifts off 50 Hz and can distort useful signal near 50 Hz. but in adaptive filtering if frequency or amplitude changes, the adaptive filter adjusts

in real-time so it doesn't distort the original signal as much because it models the noise instead of blindly filtering it out.



---

4. Research the convergence of the adaptive filtering algorithm. Explain the Gradient Descent and Stochastic Gradient Descent algorithms, and compare them based on their convergence speed.

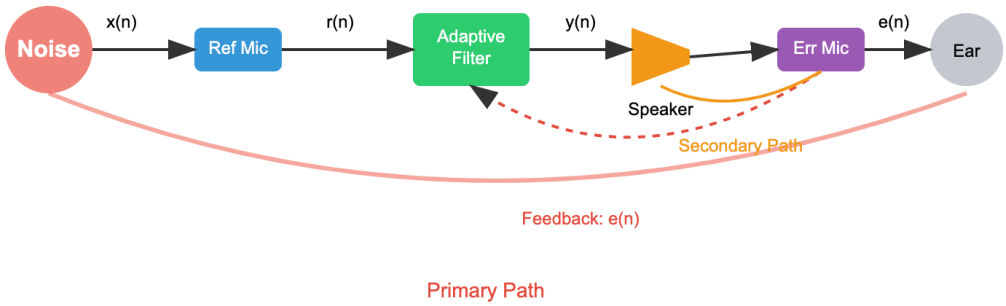
- Gradient Descent (GD): It has Update Rule: $w_{n+1} = w_n - \mu \cdot \nabla J(w_n)$ Which uses the full dataset (or full error function) to compute gradient. More accurate steps, but computationally expensive.
- Stochastic Gradient Descent (SGD): $w_{n+1} = w_n + \mu \cdot e[n] \cdot x[n]$ It uses a single sample at each step (online update). Noisy updates, but very efficient and fast

GD algorithm is slower because it requires full data per iteration. GSD has faster initial convergence, but takes noisier path.

---

5. Research Active Noise Cancelling (ANC) and explain how it can be utilized to eliminate noise from a signal. Including pictures and diagrams is strongly recommended for better illustration.

Active Noise Cancellation is a technique that uses destructive interference to cancel out unwanted noise by generating an anti-noise signal in real-time.



1. Noise Detection: Reference microphone captures incoming noise before it reaches the user
2. Signal Processing: Adaptive filter processes reference signal to predict noise at error microphone location
3. Anti-noise Generation: Speaker produces signal 180° out of phase with predicted noise
4. Destructive Interference: Anti-noise cancels original noise through wave superposition
5. Error Feedback: Error microphone measures residual noise, providing feedback for filter adaptation
6. Continuous Adaptation: System continuously updates filter coefficients to maintain optimal cancellation

1.4 Decode a Hidden Text in an Audio File Using LSB Steganography

1. Research the Least Significant Bit (LSB) method for hiding data in digital media, specifically in audio files. Why is this method useful for steganography? What are its main benefits and potential disadvantages?

What is LSB Steganography? The core idea is simple, digital audio is made up of thousands of individual *samples* per second. Each sample has a numerical value. In LSB steganography, we slightly alter the *very last bit* (the least significant bit) of some of these sample values to represent the bits (0s and 1s) of our hidden message.

Why the LSB? Because changing it has an *extremely* small effect on the overall value of the sample. For a 16-bit audio sample (which can hold values up to 32,767), changing the last bit only changes the value by 1. To the human ear, this tiny change is usually completely unnoticeable, allowing us to embed data without significantly degrading the audio quality.

Note: This project focuses on the **decoding** part – how to find and extract a message that has already been hidden inside a `.wav` file using this LSB technique.

2 Implementation

In [1]:
```
import os
import numpy as np
from scipy.io import wavfile
from scipy import signal
from scipy.fft import fft, fftfreq, ifft
from matplotlib import pyplot as plt

try:
    import seaborn as sns
    sns.set_theme()
except ImportError:
    print("Seaborn not installed, using default matplotlib style.")
    plt.style.use('default')
```

Noise cancelling:

1. Record your own voice for 10 up to 15 seconds. You can use your phone or any other recording device, but you have to turn off noise-cancelling features.

Audio File Loading `wavfile.read()`: Loads audio data and sample rate from WAV file

Stereo to Mono Conversion: Averages left and right channels if stereo signal detected

Signal Normalization: Converts to `float32` and normalizes to [-1, 1] range using `np.iinfo().max`

In [2]:
```
sample_rate, original_signal = wavfile.read('../inputs/recorded_signal.wa

# Check if the signal is stereo or mono
if len(original_signal.shape) > 1:
    original_signal = (original_signal[:, 0] + original_signal[:, 1]) / 2

# Normalize signal
original_signal = original_signal.astype(np.float32) / np.iinfo(original_
```

```
/var/folders/8l/1rmfy71x3hz98pm28kj0rrp40000gn/T/ipykernel_66145/408319814
9.py:1: WavFileWarning: Chunk (non-data) not understood, skipping it.
  sample_rate, original_signal = wavfile.read('../inputs/recorded_signal.w
av')
```

2. Add power line noise to the recorded signal. Write proper formulation for the power line noise and add it to the recorded signal.

`np.linspace()`: Creates time vector based on signal duration and sample rate

Duration Calculation: `len(signal) / sample_rate` determines total time length

Sinusoidal Noise: Generated using `np.sin(2πft + φ)` formula, Parameters:

- Frequency: 50 Hz (typical power line frequency)
- Amplitude: 0.2 (20% of signal range)
- Random phase: `np.random.uniform()` for realistic interference

Linear Addition: Simple additive noise model: `noisy_signal = original + noise`

In [3]:
```
duration = len(original_signal) / sample_rate
t = np.linspace(0, duration, len(original_signal), endpoint=False)

# Parameters for power line noise
noise_freq = 50
amplitude = 0.2
phase = np.random.uniform(0, 2*np.pi)

print(f"Adding power line noise with frequency {noise_freq} Hz, amplitude

powerline_noise = amplitude * np.sin(2 * np.pi * noise_freq * t + phase)

# Add noise to original signal
noisy_signal = original_signal + powerline_noise
```

```
Adding power line noise with frequency 50 Hz, amplitude 0.2, and phase 5.2
7 radians.
```

3. Implement the adaptive filtering algorithm to remove the power line noise from the recorded signal. Write the code in a clear and organized manner; use comments to explain the code and writing functions for building blocks is required. Use the LMS loss function and Gradient Descent algorithm.

`generate_reference_signals` creates a time vector t scaled by sample rate, Generates two reference signals:

- sine_ref: Pure sine wave at noise frequency
- cosine_ref: Pure cosine wave at same frequency

Together these form an orthogonal basis to represent any phase shift of the noise. The dual reference approach (sine + cosine) allows the system to cancel noise regardless of its phase relationship to the signal.

`lms_adaptive_filter` processes entire signal per epoch instead of sample-by-sample updates

- Weight Vector: `w = [w_sine, w_cosine]` adapts to estimate noise amplitude and phase
- Error Calculation: `e = primary_signal - estimated_noise` drives adaptation

Algorithm Steps:

1. Noise Estimation: $y = w \cdot [sine_ref, cosine_ref]$ reconstructs interference
2. Gradient Computation: $grad = \sum(e * reference_vector)/n_samples$

3. Weight Update: $w = w + \mu * avg_gradient$ using learning rate $\mu = 0.1$
4. Signal Cleaning: Subtract estimated noise from primary signal

In [4]:
```python
def generate_reference_signals(signal_length, sample_rate, noise_freq):
    """
    Generate in-phase (sine) and quadrature (cosine) reference signals
    for adaptive noise cancellation.

    Parameters:
        signal_length (int): Number of samples in the signal
        sample_rate (int): Sampling rate in Hz
        noise_freq (float): Frequency of noise to cancel (Hz)

    Returns:
        tuple: (sine_ref, cosine_ref) reference signals
    """

    t = np.arange(signal_length) / sample_rate
    sine_ref = np.sin(2 * np.pi * noise_freq * t)
    cosine_ref = np.cos(2 * np.pi * noise_freq * t)
    return sine_ref, cosine_ref

"""
This function implements an LMS adaptive filter using SGD to cancel sinus
But I keep it for reference only, as it is not used in the final implemen
"""
# def lms_adaptive_filter_SGD(primary_signal, sine_ref, cosine_ref, mu, w
#     """
#     LMS adaptive noise canceller with two weights for sinusoidal interf

#     Parameters:
#         primary_signal (ndarray): Noisy input signal (s + noise)
#         sine_ref (ndarray): Sine reference signal (in-phase component)
#         cosine_ref (ndarray): Cosine reference signal (quadrature compo
#         mu (float): Step size (learning rate)
#         w_init (tuple): Initial weights (w_sine, w_cosine)

#     Returns:
#         tuple: (cleaned_signal, weights_history, final_weights)
#     """

#     n_samples = len(primary_signal)
#     cleaned_signal = np.zeros(n_samples)
#     weights_history = np.zeros((n_samples, 2))

#     # Initialize weights
#     w = np.array(w_init) if w_init is not None else np.zeros(2)

    # for n in range(n_samples):
    #     # Create reference vector
    #     x = np.array([sine_ref[n], cosine_ref[n]])

    #     # Filter output (estimated noise)
    #     y = np.dot(w, x)

    #     # Error signal (cleaned signal estimate)
    #     e = primary_signal[n] - y
    #     cleaned_signal[n] = e
```

```
    #     # LMS weight update
    #     w = w + mu * e * x

    #     # Store weights for analysis
    #     weights_history[n] = w

#     return cleaned_signal, weights_history, w

def lms_adaptive_filter(primary_signal, sine_ref, cosine_ref, mu, w_init=
    """
    LMS adaptive filter using Gradient Descent

    Parameters:
        primary_signal: Noisy input signal
        sine_ref, cosine_ref: Reference signals
        mu: Learning rate
        w_init: Initial weights [w_sine, w_cosine]
        n_epochs: Number of full passes through data

    Returns:
        cleaned_signal: Final denoised signal
        weights_history: Weight values after each epoch
        final_weights: Learned weight vector
    """

    n_samples = len(primary_signal)
    cleaned_signal = np.zeros(n_samples)
    weights_history = []

    # Initialize weights
    w = np.array(w_init) if w_init is not None else np.zeros(2)
    weights_history.append(w.copy())

    for epoch in range(n_epochs):
        # Initialize gradient accumulation
        grad_sum = np.zeros(2)
        epoch_error = 0

        # Full pass through data
        for n in range(n_samples):
            # Reference vector
            x = np.array([sine_ref[n], cosine_ref[n]])

            # Noise estimate
            y = np.dot(w, x)

            # Error signal
            e = primary_signal[n] - y
            epoch_error += e**2

            # Gradient
            grad_sum += e * x

        # Average gradient over all samples
        avg_gradient = grad_sum / n_samples

        # Batch weight update
        w = w + mu * avg_gradient

        # Store weights for this epoch
```

```
        weights_history.append(w.copy())

        # Print progress
        mse = epoch_error / n_samples
        print(f"Epoch {epoch+1}/{n_epochs} | MSE: {mse:.6f} | Weights: [{

    # Generate final cleaned signal
    for n in range(n_samples):
        x = np.array([sine_ref[n], cosine_ref[n]])
        y = np.dot(w, x)
        cleaned_signal[n] = primary_signal[n] - y

    return cleaned_signal, np.array(weights_history), w

mu = 0.1        # Step size (learning rate)

# Generate reference signals
sine_ref, cosine_ref = generate_reference_signals(
    len(noisy_signal), sample_rate, noise_freq
)

# Apply adaptive filter
adaptive_cleaned, weights_history, final_weights = lms_adaptive_filter(
    noisy_signal, sine_ref, cosine_ref, mu
)

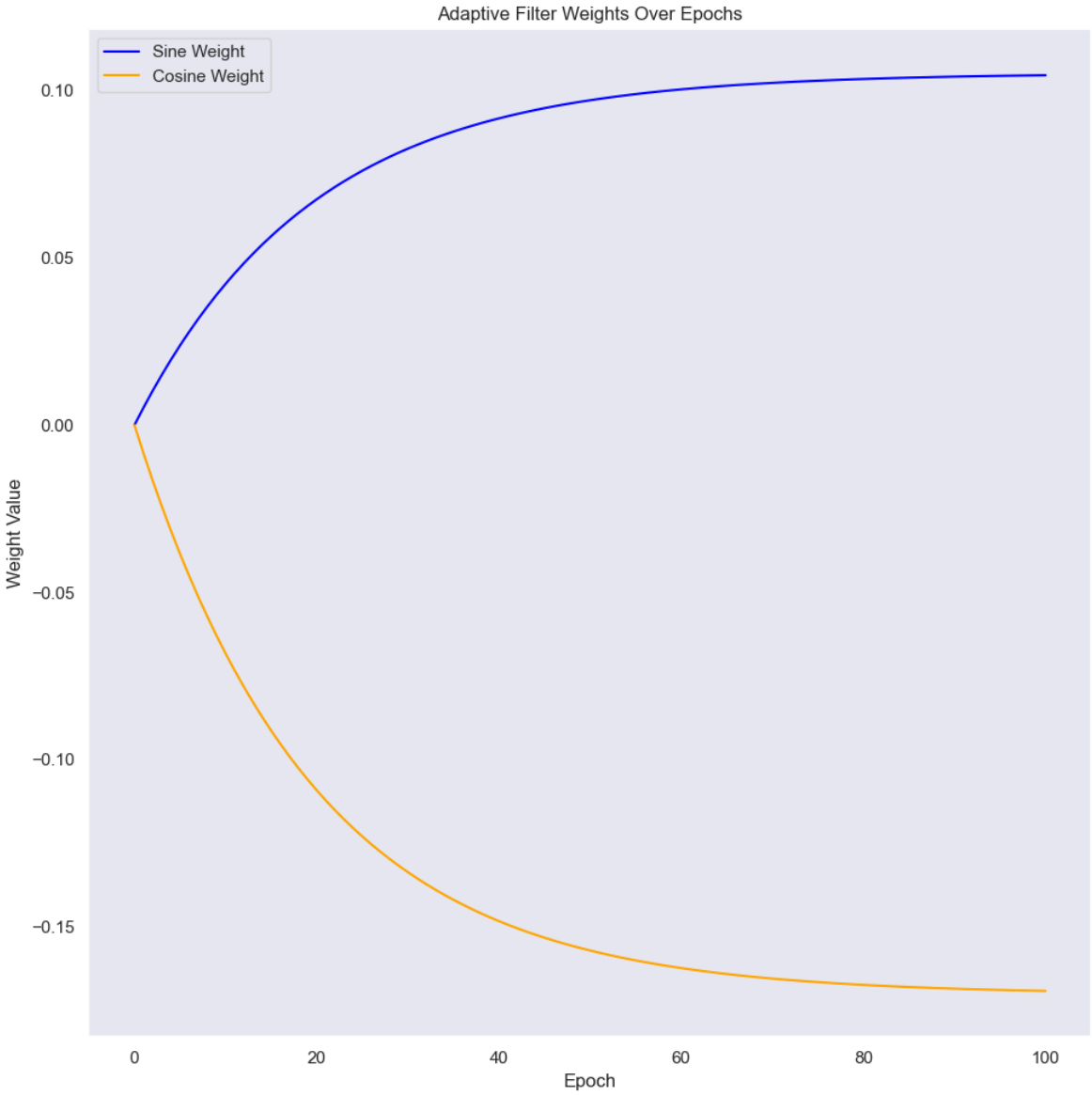

print(f"Final weights: sine={final_weights[0]:.4f}, cosine={final_weights
```

```
Epoch 1/100 | MSE: 0.021780 | Weights: [0.0053, -0.0085]
Epoch 2/100 | MSE: 0.019830 | Weights: [0.0103, -0.0166]
Epoch 3/100 | MSE: 0.018071 | Weights: [0.0150, -0.0243]
Epoch 4/100 | MSE: 0.016483 | Weights: [0.0195, -0.0316]
Epoch 5/100 | MSE: 0.015050 | Weights: [0.0238, -0.0385]
Epoch 6/100 | MSE: 0.013756 | Weights: [0.0279, -0.0451]
Epoch 7/100 | MSE: 0.012589 | Weights: [0.0317, -0.0513]
Epoch 8/100 | MSE: 0.011536 | Weights: [0.0354, -0.0573]
Epoch 9/100 | MSE: 0.010585 | Weights: [0.0389, -0.0629]
Epoch 10/100 | MSE: 0.009727 | Weights: [0.0422, -0.0683]
Epoch 11/100 | MSE: 0.008952 | Weights: [0.0453, -0.0733]
Epoch 12/100 | MSE: 0.008253 | Weights: [0.0483, -0.0782]
Epoch 13/100 | MSE: 0.007622 | Weights: [0.0512, -0.0828]
Epoch 14/100 | MSE: 0.007053 | Weights: [0.0539, -0.0871]
Epoch 15/100 | MSE: 0.006539 | Weights: [0.0564, -0.0913]
Epoch 16/100 | MSE: 0.006076 | Weights: [0.0589, -0.0952]
Epoch 17/100 | MSE: 0.005657 | Weights: [0.0612, -0.0990]
Epoch 18/100 | MSE: 0.005279 | Weights: [0.0634, -0.1025]
Epoch 19/100 | MSE: 0.004939 | Weights: [0.0655, -0.1059]
Epoch 20/100 | MSE: 0.004631 | Weights: [0.0675, -0.1091]
Epoch 21/100 | MSE: 0.004353 | Weights: [0.0693, -0.1122]
Epoch 22/100 | MSE: 0.004103 | Weights: [0.0711, -0.1151]
Epoch 23/100 | MSE: 0.003877 | Weights: [0.0728, -0.1178]
Epoch 24/100 | MSE: 0.003672 | Weights: [0.0745, -0.1204]
Epoch 25/100 | MSE: 0.003488 | Weights: [0.0760, -0.1229]
Epoch 26/100 | MSE: 0.003322 | Weights: [0.0774, -0.1253]
Epoch 27/100 | MSE: 0.003172 | Weights: [0.0788, -0.1275]
Epoch 28/100 | MSE: 0.003037 | Weights: [0.0801, -0.1296]
Epoch 29/100 | MSE: 0.002914 | Weights: [0.0814, -0.1317]
Epoch 30/100 | MSE: 0.002804 | Weights: [0.0826, -0.1336]
Epoch 31/100 | MSE: 0.002705 | Weights: [0.0837, -0.1354]
Epoch 32/100 | MSE: 0.002615 | Weights: [0.0848, -0.1372]
Epoch 33/100 | MSE: 0.002534 | Weights: [0.0858, -0.1388]
Epoch 34/100 | MSE: 0.002460 | Weights: [0.0868, -0.1404]
Epoch 35/100 | MSE: 0.002394 | Weights: [0.0877, -0.1419]
Epoch 36/100 | MSE: 0.002335 | Weights: [0.0886, -0.1433]
Epoch 37/100 | MSE: 0.002281 | Weights: [0.0894, -0.1446]
Epoch 38/100 | MSE: 0.002232 | Weights: [0.0902, -0.1459]
Epoch 39/100 | MSE: 0.002189 | Weights: [0.0909, -0.1471]
Epoch 40/100 | MSE: 0.002149 | Weights: [0.0916, -0.1482]
Epoch 41/100 | MSE: 0.002113 | Weights: [0.0923, -0.1493]
Epoch 42/100 | MSE: 0.002081 | Weights: [0.0930, -0.1504]
Epoch 43/100 | MSE: 0.002052 | Weights: [0.0936, -0.1514]
Epoch 44/100 | MSE: 0.002026 | Weights: [0.0942, -0.1523]
Epoch 45/100 | MSE: 0.002002 | Weights: [0.0947, -0.1532]
Epoch 46/100 | MSE: 0.001981 | Weights: [0.0952, -0.1540]
Epoch 47/100 | MSE: 0.001962 | Weights: [0.0957, -0.1548]
Epoch 48/100 | MSE: 0.001944 | Weights: [0.0962, -0.1556]
Epoch 49/100 | MSE: 0.001928 | Weights: [0.0966, -0.1563]
Epoch 50/100 | MSE: 0.001914 | Weights: [0.0971, -0.1570]
Epoch 51/100 | MSE: 0.001902 | Weights: [0.0975, -0.1577]
Epoch 52/100 | MSE: 0.001890 | Weights: [0.0979, -0.1583]
Epoch 53/100 | MSE: 0.001880 | Weights: [0.0982, -0.1589]
Epoch 54/100 | MSE: 0.001870 | Weights: [0.0986, -0.1594]
Epoch 55/100 | MSE: 0.001862 | Weights: [0.0989, -0.1600]
Epoch 56/100 | MSE: 0.001854 | Weights: [0.0992, -0.1605]
Epoch 57/100 | MSE: 0.001847 | Weights: [0.0995, -0.1610]
Epoch 58/100 | MSE: 0.001841 | Weights: [0.0998, -0.1614]
Epoch 59/100 | MSE: 0.001835 | Weights: [0.1001, -0.1619]
Epoch 60/100 | MSE: 0.001830 | Weights: [0.1003, -0.1623]
```

```
Epoch 61/100 | MSE: 0.001826 | Weights: [0.1006, -0.1627]
Epoch 62/100 | MSE: 0.001821 | Weights: [0.1008, -0.1630]
Epoch 63/100 | MSE: 0.001818 | Weights: [0.1010, -0.1634]
Epoch 64/100 | MSE: 0.001814 | Weights: [0.1012, -0.1637]
Epoch 65/100 | MSE: 0.001811 | Weights: [0.1014, -0.1640]
Epoch 66/100 | MSE: 0.001809 | Weights: [0.1016, -0.1643]
Epoch 67/100 | MSE: 0.001806 | Weights: [0.1018, -0.1646]
Epoch 68/100 | MSE: 0.001804 | Weights: [0.1019, -0.1649]
Epoch 69/100 | MSE: 0.001802 | Weights: [0.1021, -0.1652]
Epoch 70/100 | MSE: 0.001800 | Weights: [0.1023, -0.1654]
Epoch 71/100 | MSE: 0.001798 | Weights: [0.1024, -0.1657]
Epoch 72/100 | MSE: 0.001797 | Weights: [0.1025, -0.1659]
Epoch 73/100 | MSE: 0.001796 | Weights: [0.1027, -0.1661]
Epoch 74/100 | MSE: 0.001794 | Weights: [0.1028, -0.1663]
Epoch 75/100 | MSE: 0.001793 | Weights: [0.1029, -0.1665]
Epoch 76/100 | MSE: 0.001792 | Weights: [0.1030, -0.1667]
Epoch 77/100 | MSE: 0.001791 | Weights: [0.1031, -0.1668]
Epoch 78/100 | MSE: 0.001791 | Weights: [0.1032, -0.1670]
Epoch 79/100 | MSE: 0.001790 | Weights: [0.1033, -0.1672]
Epoch 80/100 | MSE: 0.001789 | Weights: [0.1034, -0.1673]
Epoch 81/100 | MSE: 0.001789 | Weights: [0.1035, -0.1674]
Epoch 82/100 | MSE: 0.001788 | Weights: [0.1036, -0.1676]
Epoch 83/100 | MSE: 0.001788 | Weights: [0.1037, -0.1677]
Epoch 84/100 | MSE: 0.001787 | Weights: [0.1037, -0.1678]
Epoch 85/100 | MSE: 0.001787 | Weights: [0.1038, -0.1679]
Epoch 86/100 | MSE: 0.001786 | Weights: [0.1039, -0.1680]
Epoch 87/100 | MSE: 0.001786 | Weights: [0.1039, -0.1681]
Epoch 88/100 | MSE: 0.001786 | Weights: [0.1040, -0.1682]
Epoch 89/100 | MSE: 0.001786 | Weights: [0.1041, -0.1683]
Epoch 90/100 | MSE: 0.001785 | Weights: [0.1041, -0.1684]
Epoch 91/100 | MSE: 0.001785 | Weights: [0.1042, -0.1685]
Epoch 92/100 | MSE: 0.001785 | Weights: [0.1042, -0.1686]
Epoch 93/100 | MSE: 0.001785 | Weights: [0.1043, -0.1687]
Epoch 94/100 | MSE: 0.001785 | Weights: [0.1043, -0.1687]
Epoch 95/100 | MSE: 0.001784 | Weights: [0.1044, -0.1688]
Epoch 96/100 | MSE: 0.001784 | Weights: [0.1044, -0.1689]
Epoch 97/100 | MSE: 0.001784 | Weights: [0.1044, -0.1689]
Epoch 98/100 | MSE: 0.001784 | Weights: [0.1045, -0.1690]
Epoch 99/100 | MSE: 0.001784 | Weights: [0.1045, -0.1691]
Epoch 100/100 | MSE: 0.001784 | Weights: [0.1045, -0.1691]
Final weights: sine=0.1045, cosine=-0.1691
```

In [5]:

```
fig0, ax = plt.subplots(1, 1, figsize=(10, 10))
ax.plot(weights_history[:, 0], label='Sine Weight', color='blue')
ax.plot(weights_history[:, 1], label='Cosine Weight', color='orange')
ax.set_title('Adaptive Filter Weights Over Epochs')
ax.set_xlabel('Epoch')
ax.set_ylabel('Weight Value')
ax.legend()
ax.grid()
plt.tight_layout()
```



As expected, the sine and cosine filter weight graph shows that the weight values become closer to the original values after the epochs pass and the slope of their changes becomes smaller.

4. Implement FIR and IIR filters to remove the power line noise from the recorded signal. Compare the results of the adaptive filter with the FIR and IIR filters.

`design_fir_notch()` creates a finite impulse response bandstop filter using windowing method to remove specific frequency components. Offers linear phase response but requires many coefficients for sharp frequency selectivity.

`design_iir_notch()` designs an infinite impulse response notch filter using poles and zeros placement for targeted frequency removal. Provides computational efficiency but introduces phase distortion.

Filter Design Functions

`signal.firwin()` : Window-based FIR filter design method

- Creates filter coefficients using specified window function (Blackman-Harris)

- pass_zero='bandstop' creates notch characteristics
- Window reduces spectral leakage and sidelobe levels

`signal.iirnotch()`: Designs second-order IIR notch filter

- Places zeros at target frequency and poles slightly inside unit circle
- Returns numerator (b) and denominator (a) coefficients
- Automatically handles digital frequency scaling

`signal.tf2sos()`: Converts transfer function to second-order sections

- Transforms b,a coefficients into cascaded biquad sections
- Improves numerical stability for high-order filters
- Reduces quantization errors in fixed-point implementations

Filter Application Functions

`signal.lfilter()`: Applies FIR/IIR filter using direct form

- Linear convolution for FIR (when denominator = 1.0)
- Processes signal sample-by-sample with feedback for IIR
- Maintains internal states for recursive filtering

`signal.sosfilt()`: Filters using second-order sections

- Cascades multiple biquad filters for stability
- More robust than direct form for high Q factors
- Preferred method for IIR filtering implementation

In [6]:
```
def design_fir_notch(sample_rate, noise_freq, Q, numtaps):
    """
    FIR filter design using the window method.

    Parameters:
        sample_rate (int): Sampling frequency in Hz
        noise_freq (float): Notch frequency to remove (Hz)
        Q (float): Quality factor (higher = narrower notch)
        numtaps (int): Number of filter coefficients (odd recommended)

    Returns:
        b (ndarray): FIR filter coefficients
    """

    nyquist = 0.5 * sample_rate
    width = noise_freq / Q
    bands = [max(0, noise_freq - width/2), min(nyquist, noise_freq + widt

    b = signal.firwin(numtaps,
                      bands,
                      fs=sample_rate,
                      pass_zero='bandstop',
                      window='blackmanharris')
    return b

def design_iir_notch(sample_rate, noise_freq, Q):
```

```
    """
    A second-order IIR notch digital filter.

    Parameters:
        sample_rate (int): Sampling frequency in Hz
        noise_freq (float): Notch frequency to remove (Hz)
        Q (float): Quality factor (higher = narrower notch)

    Returns:
        sos (ndarray): Second-order sections coefficients
    """

    b, a = signal.iirnotch(noise_freq, Q, fs=sample_rate)
    sos = signal.tf2sos(b, a)
    return sos

fir_coeffs = design_fir_notch(sample_rate, noise_freq, 30, 2001)
fir_cleaned = signal.lfilter(fir_coeffs, 1.0, noisy_signal)

iir_sos = design_iir_notch(sample_rate, noise_freq, 30)
iir_cleaned = signal.sosfilt(iir_sos, noisy_signal)
```

We're going to compare filters in Part 5 after ploting them.

5. Plot the original signal, the noisy signal, and the filtered signals. Write functions to play the audio signals.

In [7]:
```
# Compute FFT
N = len(noisy_signal)
xfreq = fftfreq(N, 1/sample_rate)
orig_freq = fft(original_signal)
noisy_freq = fft(noisy_signal)
adaptive_freq = fft(adaptive_cleaned)
fir_freq = fft(fir_cleaned)
iir_freq = fft(iir_cleaned)
```

`plot_time_and_freq()` : Creates dual-domain visualization (time and frequency)

- Time plot: Shows signal amplitude vs time using `ax.plot(t, signal)`
- Frequency plot: Displays magnitude spectrum using `abs(signal_freq)`
- `xlim(0, sample_rate/20)` : Limits frequency display to relevant range

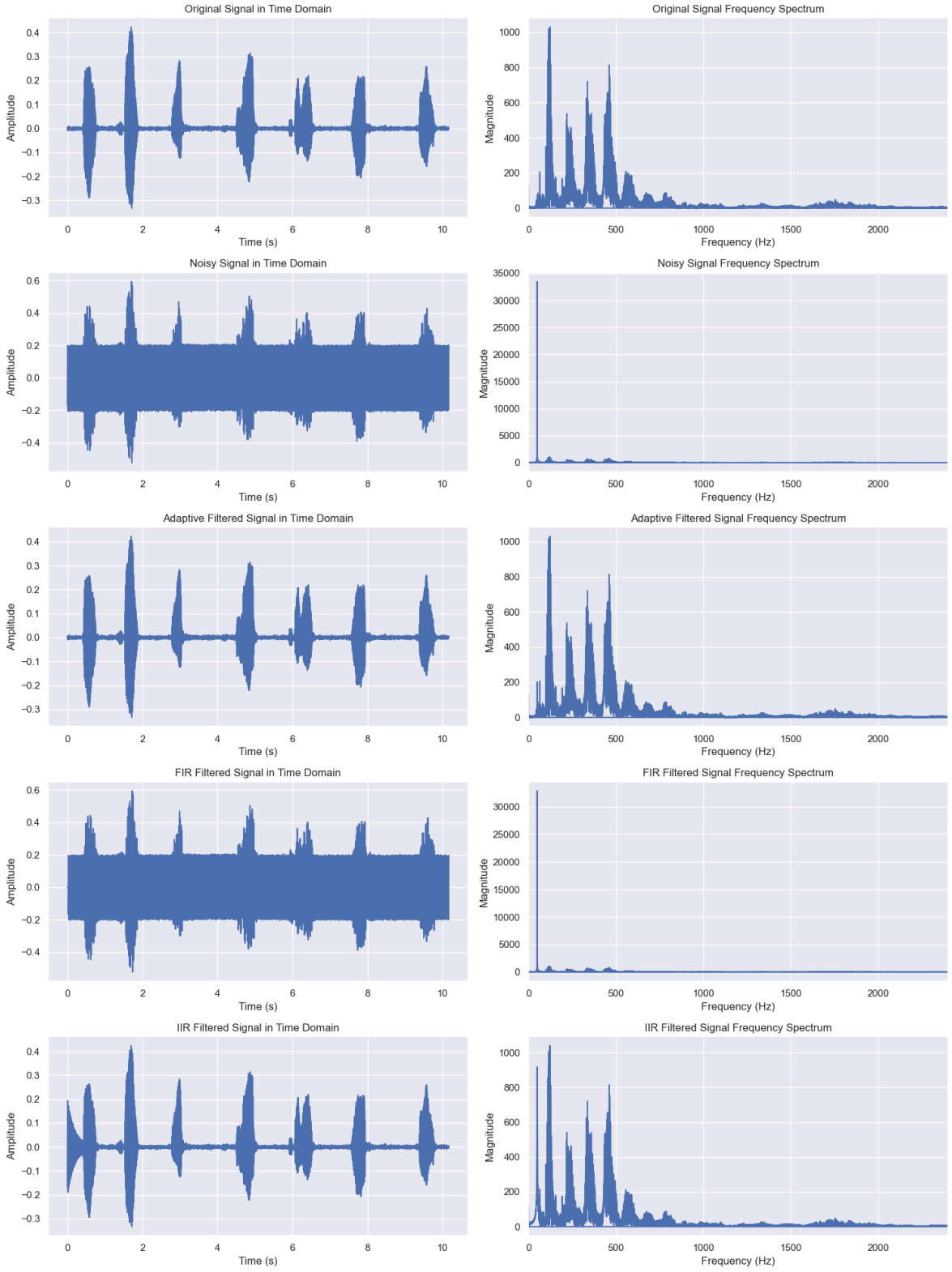
In [8]:
```
# Plot
def plot_time_and_freq(ax_row, t, signal_time, xfreq, signal_freq, title_
    # Time domain
    ax_row[0].plot(t, signal_time)
    ax_row[0].set_title(f'{title_prefix} in Time Domain')
    ax_row[0].set_xlabel('Time (s)')
    ax_row[0].set_ylabel('Amplitude')

    # Frequency domain
    ax_row[1].plot(xfreq, abs(signal_freq))
    ax_row[1].set_title(f'{title_prefix} Frequency Spectrum')
    ax_row[1].set_xlim(0, sample_rate / 20)
    ax_row[1].set_xlabel('Frequency (Hz)')
    ax_row[1].set_ylabel('Magnitude')
```

```
fig1, ax = plt.subplots(5, 2, figsize=(15, 20))

# Plot all signals
plot_time_and_freq(ax[0], t, original_signal, xfreq, orig_freq, 'Original
plot_time_and_freq(ax[1], t, noisy_signal, xfreq, noisy_freq, 'Noisy Sign
plot_time_and_freq(ax[2], t, adaptive_cleaned, xfreq, adaptive_freq, 'Ada
plot_time_and_freq(ax[3], t, fir_cleaned, xfreq, fir_freq, 'FIR Filtered
plot_time_and_freq(ax[4], t, iir_cleaned, xfreq, iir_freq, 'IIR Filtered

fig1.tight_layout()
```



Plot analysis

- Original Signal Characteristics (Row 1) In the first row displaying the original signal, individual spoken words are clearly distinguishable in the time domain plot, with the signal's frequency content concentrated primarily between 0-500 Hz, which is typical for speech signals.
- Noisy Signal Analysis (Row 2) After adding 50 Hz power line noise to the original signal, the contamination is evident in the time domain plot. The noise amplitude is clearly visible relative to the original signal, matching the configured amplitude of 0.2. In the frequency domain, a prominent peak appears at 50 Hz, representing the added interference.
- Adaptive Filter Performance (Row 3) Following application of the adaptive LMS filter, the noise has been significantly reduced, making the time domain signal closely resemble the original clean signal. In the frequency domain, the 50 Hz peak has been substantially attenuated, demonstrating effective noise cancellation.
- FIR Filter Performance (Row 4) After applying the FIR notch filter, the performance is suboptimal as this filter design does not provide satisfactory results. The noise remains clearly visible, indicating inadequate noise suppression capabilities.
- IIR Filter Performance (Row 5) The IIR notch filter successfully removes the target noise, though its performance is inferior to the adaptive filter. While effective at noise cancellation, it shows reduced performance compared to the adaptive LMS approach in terms of overall signal quality preservation.

`plot_spectrum()` : Compares multiple signals in frequency domain

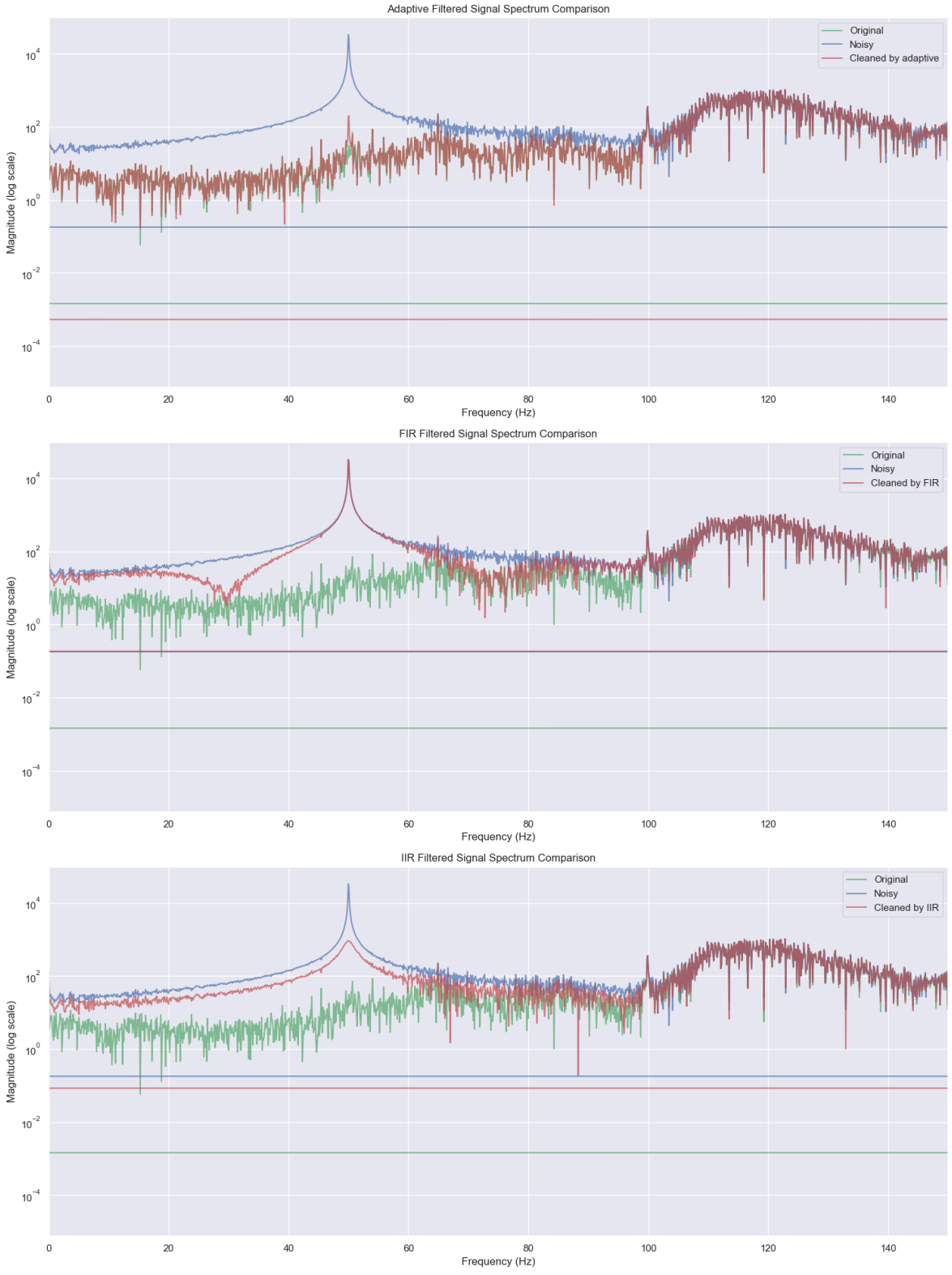
- Uses `semilogy()` for logarithmic magnitude scale
- Overlays original, noisy, and cleaned spectra with transparency (alpha=0.7)
- `xlim(0, 150)` : Focuses on low frequencies where power line noise exists

In [9]:
```
def plot_spectrum(ax, xfreq, orig_freq, noisy_freq, cleaned_freq, label,
    ax.semilogy(xfreq, abs(orig_freq), 'g', label='Original', alpha=0.7)
    ax.semilogy(xfreq, abs(noisy_freq), 'b', label='Noisy', alpha=0.7)
    ax.semilogy(xfreq, abs(cleaned_freq), 'r', label=label, alpha=0.7)
    ax.set_xlim(0, 150)
    ax.set_title(title)
    ax.set_xlabel('Frequency (Hz)')
    ax.set_ylabel('Magnitude (log scale)')
    ax.legend()
    ax.grid(True)

fig2, ax = plt.subplots(3, 1, figsize=(15, 20))

plot_spectrum(ax[0], xfreq, orig_freq, noisy_freq, adaptive_freq, 'Cleane
plot_spectrum(ax[1], xfreq, orig_freq, noisy_freq, fir_freq, 'Cleaned by
plot_spectrum(ax[2], xfreq, orig_freq, noisy_freq, iir_freq, 'Cleaned by
```

```
fig2.tight_layout()
```



- Adaptive Filter Spectrum Comparison The subsequent spectral plots demonstrate that the adaptive filter not only eliminates the 50 Hz noise component but also produces a filtered signal that closely matches the original signal's frequency characteristics across the entire spectrum.

- FIR Filter Spectrum Analysis The second comparison plot reveals poor FIR filter performance, showing minimal noise attenuation at the target frequency. The

filter introduces unwanted distortion by slightly affecting frequencies above 50 Hz while bringing some frequencies below 50 Hz closer to the original signal levels. Overall, this filter performs inadequately, causing more signal degradation than noise removal.

- IIR Filter Spectrum Evaluation The third comparison plot shows that the IIR filter demonstrates good performance in noise attenuation, successfully bringing the noisy signal's frequency components closer to those of the original signal. However, its overall performance remains inferior to the adaptive filter's capabilities in terms of spectral fidelity and noise suppression effectiveness.

To more accurately compare the filters, we used **snr** and **noise reduction** comparison criteria.

`calculate_snr()` : Computes Signal-to-Noise Ratio in dB

- Formula: $SNR = 10 * log10(signal_power/noise_power)$
- Higher values indicate better signal quality
- Uses mean squared power: `np.mean(signal**2)`

`calculate_improvement()` : Measures noise reduction effectiveness

- Compares original noise vs residual noise after filtering
- Formula: $Improvement = 10 * log10(original_noise/residual_noise)$
- Positive values indicate successful noise reduction

In [10]:
```
def calculate_snr(clean, noisy):
    """Calculate Signal-to-Noise Ratio (dB)"""
    signal_power = np.mean(clean**2)
    noise_power = np.mean((clean - noisy)**2)
    return 10 * np.log10(signal_power / noise_power)

def calculate_improvement(original, noisy, cleaned):
    """Calculate noise reduction in dB"""
    original_noise = np.mean((original - noisy)**2)
    residual_noise = np.mean((original - cleaned)**2)
    return 10 * np.log10(original_noise / residual_noise)

# Original SNR (clean signal vs noisy)
original_snr = calculate_snr(original_signal, noisy_signal)

# Results table
print("\n" + "="*60)
print("Noise Cancellation Performance Comparison")
print("="*60)
print(f"{'Method':<15} | {'Output SNR (dB)':<15} | {'Noise Reduction (dB)
print("-"*60)

def print_result(method_name, cleaned_signal):
    snr = calculate_snr(original_signal, cleaned_signal)
    improvement = calculate_improvement(original_signal, noisy_signal, cl
    print(f"{method_name:<15} | {snr:15.2f} | {improvement:20.2f}")
    return snr, improvement
```

```
print_result("Original", noisy_signal)
fir_snr, fir_impr = print_result("FIR", fir_cleaned)
iir_snr, iir_impr = print_result("IIR", iir_cleaned)
adaptive_snr, adaptive_impr = print_result("Adaptive LMS", adaptive_clean
print("="*60)

# Recommendation
best_method = np.argmax([fir_impr, iir_impr, adaptive_impr])
methods = ["FIR", "IIR", "Adaptive LMS"]
print(f"\nRecommendation: {methods[best_method]} filter provides the best
```

```
============================================================
Noise Cancellation Performance Comparison
============================================================
Method          | Output SNR (dB) | Noise Reduction (dB)
------------------------------------------------------------
Original        |          -10.50 |                 0.00
FIR             |          -11.23 |                -0.73
IIR             |            9.76 |                20.26
Adaptive LMS    |           33.98 |                44.48
============================================================

Recommendation: Adaptive LMS filter provides the best noise reduction
```

Quantitative Performance Validation Using SNR and noise reduction metrics, we can validate our observations with precise numerical evidence and provide accurate performance assessment.

- FIR Filter Performance Analysis As clearly demonstrated by the quantitative results, the FIR filter not only fails to attenuate the target noise but actually introduces additional signal distortion, degrading the original signal quality rather than improving it.
- IIR Filter Performance Analysis The IIR filter successfully removes the noise interference, showing substantial improvement in both SNR and noise reduction metrics compared to the contaminated signal.
- Adaptive Filter Superior Performance The adaptive LMS filter achieves exceptional noise reduction of 44 dB, demonstrating significantly superior performance compared to the IIR filter. This substantial noise attenuation validates the adaptive filter's effectiveness in real-time interference cancellation.

Final Recommendation Based on comprehensive quantitative analysis, the adaptive filter demonstrates the best overall performance, providing optimal noise cancellation while preserving signal integrity.

`save_wav()` : Exports processed signals as WAV files

- `np.clip(signal, -1.0, 1.0)` : Prevents clipping distortion
- Converts float32 to int16 PCM format: `(signal * 32767).astype(np.int16)`
- Saves to `"outputs/"` directory for comparison

In [11]:
```python
# Save
fig1.savefig('../outputs/ResultsInTimeAndFreq.png')
fig2.savefig('../outputs/ResultsComparedToOriginal.png')

def save_wav(filename, signal, sample_rate):
    """
    Save a floating-point signal in range [-1, 1] as a 16-bit PCM WAV fil
    """
    clipped = np.clip(signal, -1.0, 1.0)
    int16_signal = (clipped * 32767).astype(np.int16)
    wavfile.write("../outputs/" + filename, sample_rate, int16_signal)

save_wav('original_signal.wav', original_signal, sample_rate)
save_wav('noisy_signal.wav', noisy_signal, sample_rate)
save_wav('adaptive_cleaned.wav', adaptive_cleaned, sample_rate)
save_wav('fir_cleaned.wav', fir_cleaned, sample_rate)
save_wav('iir_cleaned.wav', iir_cleaned, sample_rate)
```

Decoding:

Noise-free:

1. Extract the hidden message.

The Decoding Process:

1. **Read the WAV File**: First, it opens the specified `.wav` audio file and reads its contents. It gets two key things: the *sample rate* (how many sound measurements are taken per second) and the *audio samples* themselves (the numerical data representing the sound).
2. **Ensure Mono Audio**: Many audio files are in stereo (left and right channels). To keep things simple, if the audio is stereo, the script usually just picks one channel (typically the left one) to work with.
3. **Normalize**: Often, audio samples (especially 16-bit ones) are read as integers. They might be converted to a standard floating-point range (like -1.0 to 1.0) first.
4. **Apply Scaling**: The script then multiplies the samples by a specific factor (`attenuation`). This is likely designed to **reverse** a step taken during the *encoding* process. The encoder might have scaled *down* the samples before embedding the LSBs; this scaling *up* helps ensure the integer values are in the expected range for LSB extraction.
5. **Round to Integers**: After scaling, the samples are converted back into whole numbers (integers). This is essential because LSB operations are performed on the binary (integer) representation of the numbers.
6. **Select Specific Samples**: This is a crucial detail! The message isn't hidden in *every single* sample. Instead, the script starts at a particular sample (e.g., the

2625th) and then jumps forward by a fixed interval (`sample_multiple`) to pick the next sample. This means the hidden bits are spread out, making them harder to find and less likely to create noticeable patterns. *You must know this interval to decode the message correctly.*

7. **Extract LSBs**: For each *selected* sample, the script looks at its binary representation and picks out only the very last bit (the LSB). It does this using a bitwise `AND` operation with 1. If the LSB is 1, the result is 1; if it's 0, the result is 0. These extracted 0s and 1s are the raw bits of our hidden message.

In [12]:
```python
def decode_lsb_from_wav(filename, sample_multiple=2625, attenuation=100):

    try:
        sample_rate, samples = wavfile.read(filename)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        return ""

    # If stereo, take only the first channel
    if samples.ndim > 1:
        samples = samples[:, 0]

    # Normalize samples
    if samples.dtype == np.int16:
        samples = samples / 32768.0  # Max value for int16
    elif samples.dtype == np.int32:
        samples = samples / 2147483648.0 # Max value for int32

    # Apply attenuation
    samples = samples * attenuation

    # Round to nearest integer
    samples = np.round(samples).astype(int)

    # Select every Nth sample, starting from the (N-1)th index
    if sample_multiple > 0:
        selected_samples = samples[(sample_multiple-1)::sample_multiple]
    else:
        print("Error: sample_multiple must be greater than 0.")
        return ""

    # Extract the LSB (0 or 1) from each selected sample
    lsb_bits = (selected_samples & 1)

    # Convert the extracted bits to text using the new function
    hidden_message = bits_to_text(lsb_bits)

    return hidden_message

def decode_lsb_from_signal(samples, sample_multiple=2625, attenuation=100

    # If stereo, take only the first channel
    if samples.ndim > 1:
        samples = samples[:, 0]

    # Normalize samples
    if samples.dtype == np.int16:
        samples = samples / 32768.0  # Max value for int16
```

```
    elif samples.dtype == np.int32:
        samples = samples / 2147483648.0 # Max value for int32

    # Apply attenuation
    samples = samples * attenuation

    # Round to nearest integer
    samples = np.round(samples).astype(int)

    # Select every Nth sample, starting from the (N-1)th index
    if sample_multiple > 0:
        selected_samples = samples[(sample_multiple-1)::sample_multiple]
    else:
        print("Error: sample_multiple must be greater than 0.")
        return ""

    # Extract the LSB (0 or 1) from each selected sample
    lsb_bits = (selected_samples & 1)

    # Convert the extracted bits to text using the new function
    hidden_message = bits_to_text(lsb_bits)

    return hidden_message

def bits_to_text(lsb_bits):

    lsb_bits = lsb_bits.astype(str)

    message_bytes = []
    # Iterate over the bits in chunks of 8 to form bytes
    for i in range(0, len(lsb_bits), 8):
        byte_bits = lsb_bits[i:i+8]
        # Ensure we have a full byte
        if len(byte_bits) < 8:
            break
        # Join the bits in the byte and convert to an integer
        byte_str = ''.join(byte_bits)
        message_bytes.append(int(byte_str, 2))

    # Convert byte integers to characters
    hidden_message = ''.join([chr(b) for b in message_bytes])

    # Find the null terminator and truncate the message if found
    null_index = hidden_message.find('\x00')
    if null_index != -1:
        hidden_message = hidden_message[:null_index]

    return hidden_message
```

The Translator: How Turn Bits into Text

1. **Reconstruct Bytes**: Computers understand data in groups of 8 bits, called *bytes*. The script takes the long sequence of extracted LSBs and groups them into chunks of 8.

2. **Convert Bytes to Characters**: Each 8-bit byte corresponds to a character. The script converts each byte back into its character equivalent.

3. **Assemble & Clean Up**: All the characters are joined together to form the hidden message. Often, a special "null character" (`\x00`) is used to mark the *end* of the hidden message. The script looks for this null character and cuts off anything that might come after it, ensuring we only get the intended message.

Result:

In [13]:
```
hidden_message = decode_lsb_from_wav("../inputs/100-2625.wav")

if hidden_message:
    print("Hidden message:", hidden_message)
else:
    print("No message decoded or an error occurred.")
```

```
Hidden message: Signals & System
```

Noisy audio:

1. Plot the signal in the frequency domain. Determine the noise frequencies in the audio by performing an analysis of its frequency spectrum using FFT.

Analyzing Noise in a WAV File:

1. **Read the WAV File**: First, it opens the specified `.wav` audio file and reads its contents. It gets two key things: the *sample rate* (how many sound measurements are taken per second) and the *audio samples* themselves (the numerical data representing the sound).

2. **Ensure Mono Audio**: Many audio files are in stereo (left and right channels). To keep things simple, if the audio is stereo, the script usually just picks one channel (typically the left one) to work with.

3. **Transform to the Frequency Domain (FFT)**: The core of the analysis involves converting the audio signal from the time domain (how amplitude changes over time) to the frequency domain (which frequencies are present and how strong they are). This is achieved using the Fast Fourier Transform (FFT). The `fft()` function takes the audio samples and outputs a set of complex numbers. Each complex number corresponds to a specific frequency and encodes both the strength (magnitude) and phase of that frequency component in the original signal.

4. **Calculate Frequency Magnitudes & Normalize**: From the complex FFT output, we're primarily interested in the magnitude (or strength) of each frequency. This is calculated by taking the absolute value of each complex number. Crucially, these raw magnitudes are then normalized by dividing them by the total number of samples in the audio. This step is important because the raw FFT output scales with the length of the signal. Normalization makes the magnitude values independent of the audio clip's duration, allowing for more consistent comparisons and a better representation of the actual amplitude of frequency components.

5. **Determine Corresponding Frequencies**: The FFT output gives us magnitudes, but we also need to know which actual frequency (in Hz) each magnitude corresponds to. The `fftfreq()` function is used for this. It generates an array of frequency values that align with the FFT output, based on the total number of samples and the original sample rate.

6. **Focus on Positive Frequencies**: For real-world audio signals, the FFT output is symmetrical: the information in negative frequencies mirrors that in positive frequencies. Therefore, we typically only need to analyze one half of the spectrum. The code selects the portion of the frequencies and fft_magnitude arrays that correspond to 0 Hz and positive frequencies.

7. **Identify Dominant Peaks (Potential Noise)**: The function assumes that the most prominent "noise" will appear as the highest peaks in the frequency spectrum. It sets a target (num_peaks_to_find, e.g., 2) for how many of the loudest frequencies to identify. It then finds the indices of these highest magnitude peaks in the positive_fft_magnitude array.

8. **Extract Noise Frequencies**: Using the indices of the identified peaks, the function looks up the corresponding actual frequency values (in Hz) from the positive_frequencies array. These are considered the detected noise frequencies.

In [14]:
```
def analysis_noisy_Wav(filename):

    sample_rate, samples = wavfile.read(filename)

    if samples.ndim > 1:
        samples = samples[:, 0]

    fft_output = fft(samples)
    fft_magnitude = np.abs(fft_output) / len(samples)
    frequencies = fftfreq(len(samples), 1 / sample_rate)

    positive_frequencies = frequencies[:len(frequencies)//2]
    positive_fft_magnitude = fft_magnitude[:len(fft_magnitude)//2]

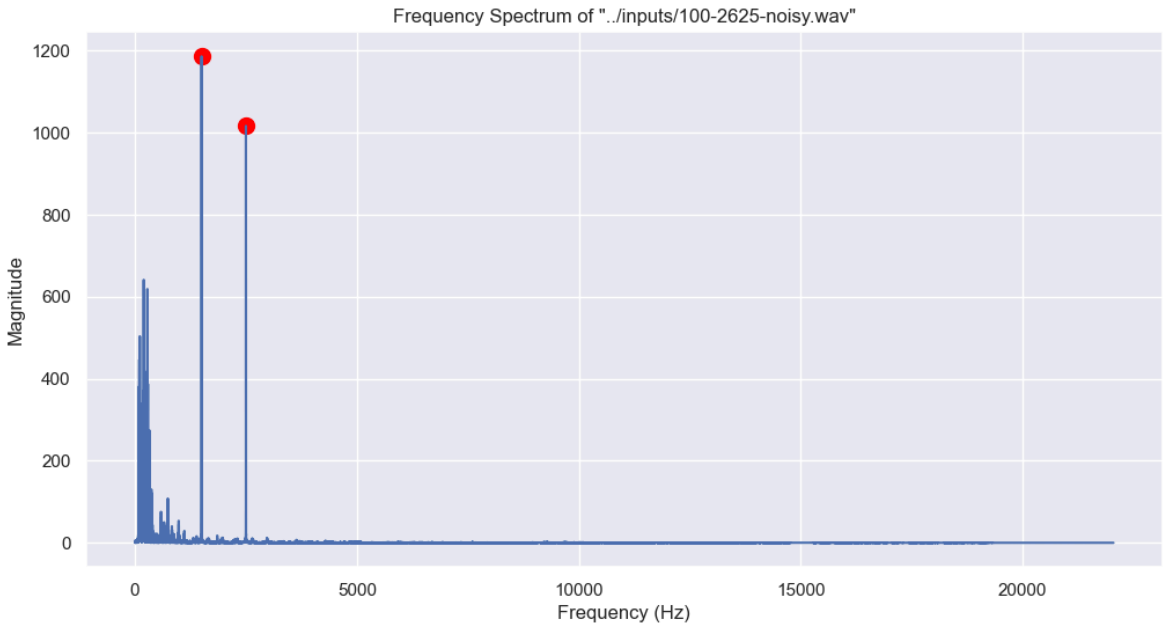
    num_peaks_to_find = 2
    peak_indices = np.argsort(positive_fft_magnitude)[-num_peaks_to_find:
    noise_frequencies_found = positive_frequencies[peak_indices]
    print(f"Detected Noise Frequencies: {noise_frequencies_found[0]} Hz a


    plt.figure(figsize=(12, 6))
    plt.plot(positive_frequencies, positive_fft_magnitude)
    plt.scatter(noise_frequencies_found, positive_fft_magnitude[peak_indi
    plt.title(f'Frequency Spectrum of "{filename}"')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.grid(True)
    plt.show()
```

Result:

In [15]:
```
analysis_noisy_Wav("../inputs/100-2625-noisy.wav")
```

```
Detected Noise Frequencies: 2500.05 Hz and 1500.06 Hz
```



2. Based on the previous section (noise cancelling), design a filter to remove the noise effectively. Note that you must adjust the order or quality factor of your filter in such a way that the samples remain unchanged. By comparing the FFT of the signal before and after filtering, you can design the filter effectively. After applying the filter, plot the signal in both the time and frequency domains to ensure that the noise has been effectively removed. If you are using a filter that was not mentioned in the previous section, be sure to first include an explanation of that filter in your report file.

`plot_fft` Function Explanation

Purpose:

The `plot_fft` function is designed to visualize the Fast Fourier Transform (FFT) spectrum of a signal.

Why It Wasn't Used in the Past?

Previously, we avoided using `plot_fft` in certain processing pipelines because:

1. The signal was already in the frequency domain (FFT had already been applied), Applying IFFT to get back to time domain just to use `plot_fft` would mean: First IFFT to convert to time domain, Then FFT again inside `plot_fft` to get back to frequency domain. This double transformation was computationally inefficient
2. We need to plt other information in same figure

In [16]:
```python
def plot_fft(samples, sample_rate, plot_title, x_max_val=None):

    fft_output = fft(samples)
    fft_magnitude = np.abs(fft_output) / len(samples)
    frequencies = fftfreq(len(samples), 1 / sample_rate)

    positive_frequencies = frequencies[:len(frequencies)//2]
    positive_fft_magnitude = fft_magnitude[:len(fft_magnitude)//2]

    plt.figure(figsize=(12, 6))
    plt.plot(positive_frequencies, positive_fft_magnitude)
    plt.title(f'Frequency Spectrum of "{plot_title}"')
    plt.xlabel('Frequency (Hz)')
    plt.ylabel('Magnitude')
    plt.grid(True)
    plt.xlim(0, x_max_val if x_max_val else sample_rate / 2)
    plt.show()
```

`find_closest_bin` Function

Purpose:

Finds the frequency bin in a frequency array that is closest to a target frequency value.

How it works:

1. Calculates the absolute difference between each frequency in `freqs_array` and the `target_freq_value`
2. Returns the index (bin number) of the smallest difference using `np.argmin()`

Mathematically:

Returns `i` such that `|freqs_array[i] - target_freq_value|` is minimized

In [17]:
```python
def find_closest_bin(freqs_array, target_freq_value):
    return np.argmin(np.abs(freqs_array - target_freq_value))
```

In [18]:
```python
def load_and_prepare_audio(filename):

    try:
        sample_rate, samples = wavfile.read(filename)
    except FileNotFoundError:
        print(f"Error: The file '{filename}' was not found.")
        return ""

    if samples.ndim > 1:
        samples = samples[:, 0]

    if samples.dtype == np.int16:
        samples = samples / 32768.0  # Max value for int16
    elif samples.dtype == np.int32:
        samples = samples / 2147483648.0 # Max value for int32

    samples_len = len(samples)
```

```
    return sample_rate, samples, samples_len
```

`perform_fft_analysis` & `perform_ifft_and_get_real` Function

frequency_resolution:

```
Low Resolution (10Hz)       High Resolution (1Hz)
|--------|--------|         |-|-|-|-|-|-|-|-|-|-|
0 Hz    10 Hz    20 Hz      0 1 2 3 ... 9 10 Hz
```

In [19]:
```
def perform_fft_analysis(samples_to_fft, samples_rate):
    fft_spectrum = fft(samples_to_fft)
    fft_frequencies = fftfreq(len(samples_to_fft), 1 / samples_rate)
    frequency_resolution = samples_rate / len(samples_to_fft)
    return fft_spectrum, fft_frequencies, frequency_resolution

def perform_ifft_and_get_real(processed_fft_spectrum):
    time_domain_complex = ifft(processed_fft_spectrum)
    time_domain_real = np.real(time_domain_complex)
    return time_domain_real
```

Noise Removal in Frequency Spectrum using `linear interpolation`

Purpose

This function removes specific noise frequencies from an FFT spectrum by replacing them with smoothly interpolated values from nearby clean frequencies.

How It Works?

1. Preparation
    - Makes a copy of the original spectrum to avoid modifying the input data
    - Calculates how many frequency bins need to be replaced based on:
        - `noise_window_hz` : How wide the noise is (in Hz)
        - `freq_res` : The frequency resolution (Hz per bin)
2. For Each Target Noise Frequency For every noise frequency you want to remove:

    a) Find the Noise

    - Locates the exact frequency bin where the noise is centered
    - Calculates a window of bins around it that will be replaced

    b) Find Clean Reference Points

    - Identifies "clean" bins just outside the noise window to use as reference points

    c) Replace the Noise

    - Performs linear interpolation between the clean reference points

- Replaces all bins in the noise window with these interpolated values
- Also updates the corresponding "mirror" bins for negative frequencies

In [20]:
```
def interpolate_noise_in_spectrum(fft_spectrum_data,
                                  fft_frequencies_data,
                                  num_total_samples,
                                  target_noise_freqs,
                                  noise_window,
                                  freq_res):

    fft_processed = np.copy(fft_spectrum_data)

    num_window_bins = int(np.ceil(noise_window / freq_res))
    if num_window_bins == 0: num_window_bins = 1 # ensure it's not zero
    if num_window_bins % 2 == 0: num_window_bins += 1 # ensure it's odd
    half_bin_window = num_window_bins // 2

    for f_noise_target in target_noise_freqs:

        k_noise_center_idx = find_closest_bin(fft_frequencies_data, f_noi

        # Define the window of bins to be replaced
        start_bin_to_replace_idx = k_noise_center_idx - half_bin_window -
        end_bin_to_replace_idx = k_noise_center_idx + half_bin_window + 1

        # Define source points for interpolation (outside the window bein
        interp_source_lower_idx = start_bin_to_replace_idx - 1
        interp_source_upper_idx = end_bin_to_replace_idx + 1

        val_interp_start = fft_spectrum_data[interp_source_lower_idx]
        val_interp_end = fft_spectrum_data[interp_source_upper_idx]

        # Perform Linear Interpolation over the window to be replaced
        for i in range(num_window_bins):
            current_bin_idx_to_replace = start_bin_to_replace_idx + i

            alpha = (i + 1.0) / (num_window_bins + 1.0)
            interpolated_value = val_interp_start * (1 - alpha) + val_int

            fft_processed[current_bin_idx_to_replace] = interpolated_valu

            # Apply conjugate to symmetric negative frequency bin
            if current_bin_idx_to_replace != 0 and current_bin_idx_to_rep
                symmetric_bin_idx = (num_total_samples - current_bin_idx
                fft_processed[symmetric_bin_idx] = np.conj(interpolated_

    return fft_processed
```

3. Extract the hidden message.

In [21]:
```
def process_audio_for_noise_cancellation(filename,
                                         noise_frequencies_list,
                                         filter_window_hz=10.0,
                                         max_freq_display=None):

    # 1. Load and Prepare Audio
    sample_rate, samples, n_samples = load_and_prepare_audio(filename)
```

```
    # 2. Plot Original Spectrum
    plot_fft(samples, sample_rate,
                     f"FFT of Original Signal ({os.path.basename(filenam
                     max_freq_display)

    # 3. Perform FFT Analysis
    original_fft_spectrum, fft_freqs, freq_res = perform_fft_analysis(sam

    # 4. Interpolate Noise Frequencies
    cleaned_fft_spectrum = interpolate_noise_in_spectrum(original_fft_spe
                                                         noise_frequencie

    # 5. Perform IFFT
    cleaned_samples_real_time = perform_ifft_and_get_real(cleaned_fft_spe

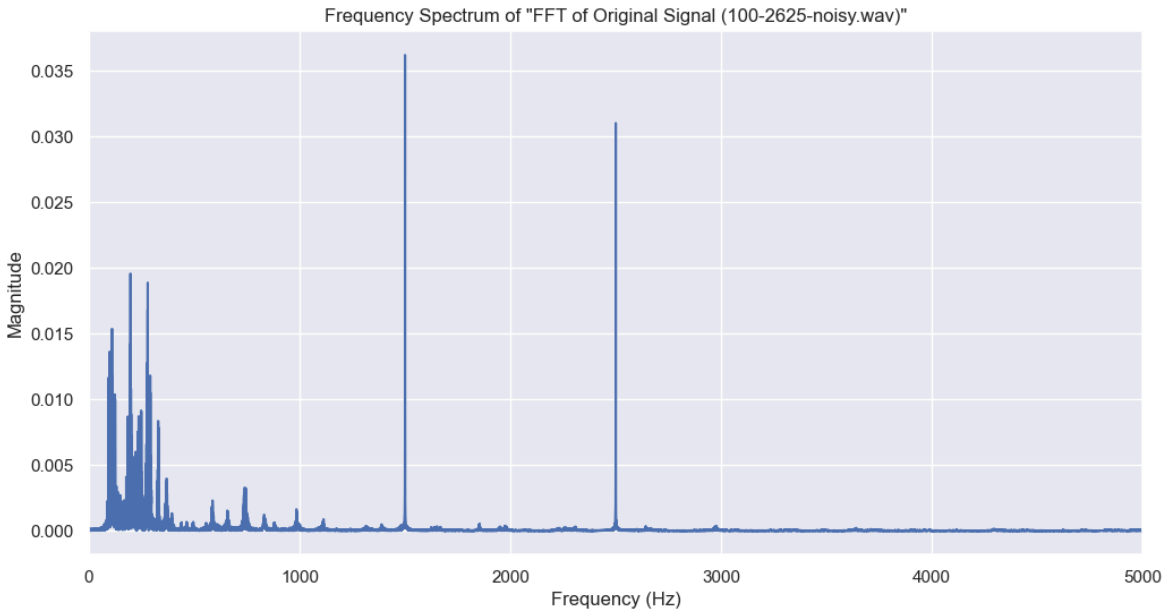
    # 6. Plot Cleaned Spectrum
    plot_fft(cleaned_samples_real_time, sample_rate,
                     f"FFT of Cleaned Signal from ({os.path.basename(fil
                     max_freq_display)

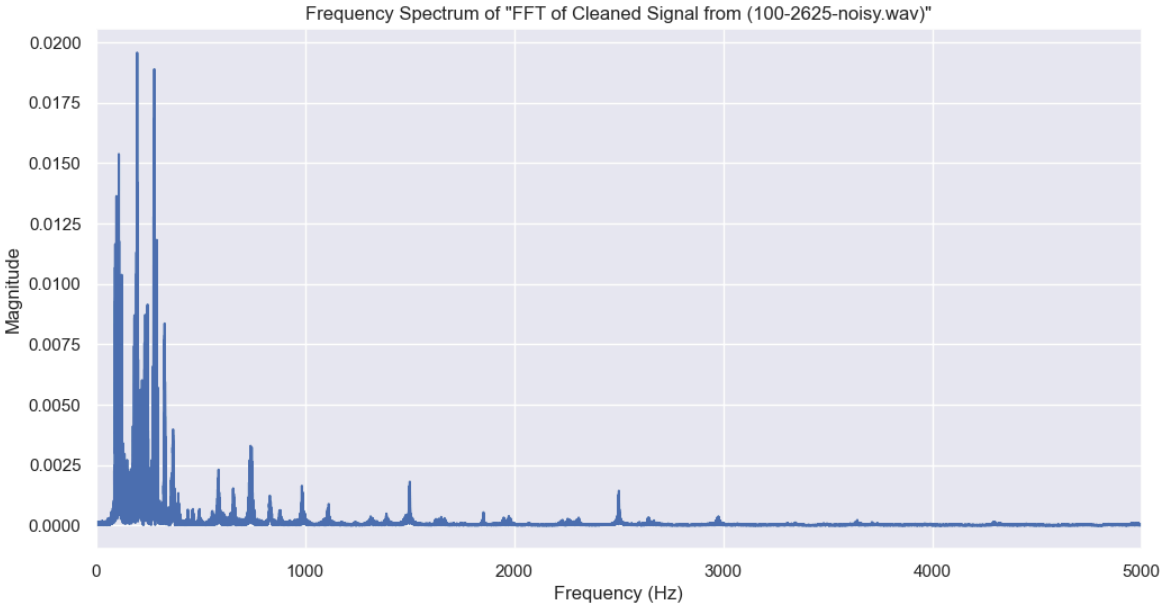
    return sample_rate, cleaned_samples_real_time

returned_sample_rate, returned_cleaned_audio = process_audio_for_noise_ca
    '../inputs/100-2625-noisy.wav',
    [1500.06, 2500.05],
    2,  # Using a very narrow window
    5000.0  # Zoom in on FFT
)

hidden_message = decode_lsb_from_signal(returned_cleaned_audio)

if hidden_message:
    print("Hidden message:", hidden_message)
else:
    print("No message decoded or an error occurred.")
```






```
Hidden message: Oignals & Syste
```

Noise Removal in Frequency Spectrum using `lms_adaptive_filter` and `iir_notch` - FAILED!

In [22]:
```
sample_rate, noisy_signal = wavfile.read("../inputs/100-2625-noisy.wav")
```

In [23]:
```
for q in range(20, 81, 5):

    iir_sos = design_iir_notch(sample_rate, 2500.05, q)
    iir_cleaned_code = signal.sosfilt(iir_sos, noisy_signal)

    iir_sos = design_iir_notch(sample_rate, 1500.05, q)
    iir_cleaned_code = signal.sosfilt(iir_sos, iir_cleaned_code)

    print(f"hidden message using iir_notch Q={q}:{decode_lsb_from_signal(
```

```
hidden message using iir_notch Q=20:  As  N±ÅnF¶6*ð Õ   - FAILED!
hidden message using iir_notch Q=25:æ  Â»\b4]E   ú    áR - FAILED!
hidden message using iir_notch Q=30:#½`Ï¿³ÉÍhaG «  - FAILED!
hidden message using iir_notch Q=35:½ ¡¡zAPÝÛ^ñAV´ - FAILED!
hidden message using iir_notch Q=40:  en£Õ1è ë·´°  r§ï - FAILED!
hidden message using iir_notch Q=45:¬þÊ  ë  ¢H" £¹ - FAILED!
hidden message using iir_notch Q=50:  J DµØp$¡  ±EA - FAILED!
hidden message using iir_notch Q=55:UÔB    â±Ôú¤  D Ô  à - FAILED!
hidden message using iir_notch Q=60: ] - FAILED!
hidden message using iir_notch Q=65:ÓÁMF  Ë–¦  u    ±>7J - FAILED!
hidden message using iir_notch Q=70:ÖâàÉÕOóÙÛÎ  ®oÈ¿   - FAILED!
hidden message using iir_notch Q=75:gîÛ(@¨w  ðV Õ¢£ - FAILED!
hidden message using iir_notch Q=80:%   x  ù tÓÛv)U[µ( - FAILED!
```

In [24]:
```
mu = 0.005 # Step size (learning rate)

sine_ref, cosine_ref = generate_reference_signals(
    len(noisy_signal), sample_rate, 1500.06
)

# Apply adaptive filter
adaptive_cleaned_code, weights_history, final_weights = lms_adaptive_filt
    noisy_signal, sine_ref, cosine_ref, mu, n_epochs=200
```

```
)

sine_ref, cosine_ref = generate_reference_signals(
    len(adaptive_cleaned_code), sample_rate, 2500.05
)

# Apply adaptive filter
adaptive_cleaned_code, weights_history, final_weights = lms_adaptive_filt
    adaptive_cleaned_code, sine_ref, cosine_ref, mu, n_epochs=200
)

print(f"hidden message using lms_adaptive_filter : {decode_lsb_from_signa
```

```
Epoch 1/200 | MSE: 51117463.025244 | Weights: [0.7700, -5.5989]
Epoch 2/200 | MSE: 51104702.857497 | Weights: [1.5381, -11.1838]
Epoch 3/200 | MSE: 51092006.410990 | Weights: [2.3043, -16.7547]
Epoch 4/200 | MSE: 51079373.367518 | Weights: [3.0685, -22.3117]
Epoch 5/200 | MSE: 51066803.410460 | Weights: [3.8309, -27.8548]
Epoch 6/200 | MSE: 51054296.224779 | Weights: [4.5913, -33.3840]
Epoch 7/200 | MSE: 51041851.497008 | Weights: [5.3498, -38.8994]
Epoch 8/200 | MSE: 51029468.915247 | Weights: [6.1065, -44.4010]
Epoch 9/200 | MSE: 51017148.169154 | Weights: [6.8612, -49.8889]
Epoch 10/200 | MSE: 51004888.949936 | Weights: [7.6141, -55.3631]
Epoch 11/200 | MSE: 50992690.950343 | Weights: [8.3651, -60.8235]
Epoch 12/200 | MSE: 50980553.864659 | Weights: [9.1142, -66.2704]
Epoch 13/200 | MSE: 50968477.388693 | Weights: [9.8614, -71.7036]
Epoch 14/200 | MSE: 50956461.219779 | Weights: [10.6068, -77.1232]
Epoch 15/200 | MSE: 50944505.056750 | Weights: [11.3503, -82.5292]
Epoch 16/200 | MSE: 50932608.599957 | Weights: [12.0919, -87.9218]
Epoch 17/200 | MSE: 50920771.551239 | Weights: [12.8317, -93.3009]
Epoch 18/200 | MSE: 50908993.613927 | Weights: [13.5696, -98.6665]
Epoch 19/200 | MSE: 50897274.492833 | Weights: [14.3057, -104.0187]
Epoch 20/200 | MSE: 50885613.894242 | Weights: [15.0400, -109.3575]
Epoch 21/200 | MSE: 50874011.525906 | Weights: [15.7724, -114.6830]
Epoch 22/200 | MSE: 50862467.097038 | Weights: [16.5029, -119.9952]
Epoch 23/200 | MSE: 50850980.318304 | Weights: [17.2317, -125.2941]
Epoch 24/200 | MSE: 50839550.901808 | Weights: [17.9586, -130.5797]
Epoch 25/200 | MSE: 50828178.561101 | Weights: [18.6838, -135.8521]
Epoch 26/200 | MSE: 50816863.011156 | Weights: [19.4071, -141.1114]
Epoch 27/200 | MSE: 50805603.968379 | Weights: [20.1286, -146.3575]
Epoch 28/200 | MSE: 50794401.150583 | Weights: [20.8483, -151.5905]
Epoch 29/200 | MSE: 50783254.276992 | Weights: [21.5661, -156.8104]
Epoch 30/200 | MSE: 50772163.068238 | Weights: [22.2822, -162.0172]
Epoch 31/200 | MSE: 50761127.246343 | Weights: [22.9966, -167.2111]
Epoch 32/200 | MSE: 50750146.534717 | Weights: [23.7091, -172.3919]
Epoch 33/200 | MSE: 50739220.658153 | Weights: [24.4198, -177.5598]
Epoch 34/200 | MSE: 50728349.342817 | Weights: [25.1288, -182.7148]
Epoch 35/200 | MSE: 50717532.316245 | Weights: [25.8360, -187.8569]
Epoch 36/200 | MSE: 50706769.307330 | Weights: [26.5414, -192.9861]
Epoch 37/200 | MSE: 50696060.046322 | Weights: [27.2451, -198.1025]
Epoch 38/200 | MSE: 50685404.264816 | Weights: [27.9470, -203.2061]
Epoch 39/200 | MSE: 50674801.695748 | Weights: [28.6471, -208.2970]
Epoch 40/200 | MSE: 50664252.073388 | Weights: [29.3455, -213.3751]
Epoch 41/200 | MSE: 50653755.133334 | Weights: [30.0422, -218.4406]
Epoch 42/200 | MSE: 50643310.612501 | Weights: [30.7371, -223.4933]
Epoch 43/200 | MSE: 50632918.249120 | Weights: [31.4302, -228.5335]
Epoch 44/200 | MSE: 50622577.782731 | Weights: [32.1217, -233.5610]
Epoch 45/200 | MSE: 50612288.954170 | Weights: [32.8114, -238.5760]
Epoch 46/200 | MSE: 50602051.505572 | Weights: [33.4994, -243.5784]
Epoch 47/200 | MSE: 50591865.180358 | Weights: [34.1856, -248.5684]
Epoch 48/200 | MSE: 50581729.723230 | Weights: [34.8702, -253.5458]
Epoch 49/200 | MSE: 50571644.880163 | Weights: [35.5530, -258.5108]
Epoch 50/200 | MSE: 50561610.398403 | Weights: [36.2342, -263.4634]
Epoch 51/200 | MSE: 50551626.026460 | Weights: [36.9136, -268.4036]
Epoch 52/200 | MSE: 50541691.514093 | Weights: [37.5913, -273.3315]
Epoch 53/200 | MSE: 50531806.612320 | Weights: [38.2673, -278.2470]
Epoch 54/200 | MSE: 50521971.073392 | Weights: [38.9417, -283.1503]
Epoch 55/200 | MSE: 50512184.650810 | Weights: [39.6144, -288.0413]
Epoch 56/200 | MSE: 50502447.099291 | Weights: [40.2853, -292.9201]
Epoch 57/200 | MSE: 50492758.174791 | Weights: [40.9546, -297.7867]
Epoch 58/200 | MSE: 50483117.634474 | Weights: [41.6223, -302.6411]
Epoch 59/200 | MSE: 50473525.236722 | Weights: [42.2882, -307.4833]
Epoch 60/200 | MSE: 50463980.741123 | Weights: [42.9525, -312.3135]
```

```
Epoch 61/200 | MSE: 50454483.908465 | Weights: [43.6151, -317.1316]
Epoch 62/200 | MSE: 50445034.500731 | Weights: [44.2761, -321.9376]
Epoch 63/200 | MSE: 50435632.281090 | Weights: [44.9354, -326.7317]
Epoch 64/200 | MSE: 50426277.013897 | Weights: [45.5931, -331.5137]
Epoch 65/200 | MSE: 50416968.464685 | Weights: [46.2492, -336.2838]
Epoch 66/200 | MSE: 50407706.400152 | Weights: [46.9035, -341.0420]
Epoch 67/200 | MSE: 50398490.588170 | Weights: [47.5563, -345.7882]
Epoch 68/200 | MSE: 50389320.797758 | Weights: [48.2074, -350.5226]
Epoch 69/200 | MSE: 50380196.799099 | Weights: [48.8569, -355.2452]
Epoch 70/200 | MSE: 50371118.363517 | Weights: [49.5048, -359.9560]
Epoch 71/200 | MSE: 50362085.263486 | Weights: [50.1510, -364.6550]
Epoch 72/200 | MSE: 50353097.272606 | Weights: [50.7957, -369.3422]
Epoch 73/200 | MSE: 50344154.165616 | Weights: [51.4387, -374.0177]
Epoch 74/200 | MSE: 50335255.718373 | Weights: [52.0801, -378.6815]
Epoch 75/200 | MSE: 50326401.707859 | Weights: [52.7199, -383.3337]
Epoch 76/200 | MSE: 50317591.912170 | Weights: [53.3581, -387.9743]
Epoch 77/200 | MSE: 50308826.110502 | Weights: [53.9948, -392.6032]
Epoch 78/200 | MSE: 50300104.083166 | Weights: [54.6298, -397.2206]
Epoch 79/200 | MSE: 50291425.611559 | Weights: [55.2632, -401.8264]
Epoch 80/200 | MSE: 50282790.478172 | Weights: [55.8951, -406.4207]
Epoch 81/200 | MSE: 50274198.466591 | Weights: [56.5254, -411.0035]
Epoch 82/200 | MSE: 50265649.361470 | Weights: [57.1541, -415.5749]
Epoch 83/200 | MSE: 50257142.948546 | Weights: [57.7812, -420.1348]
Epoch 84/200 | MSE: 50248679.014626 | Weights: [58.4068, -424.6834]
Epoch 85/200 | MSE: 50240257.347578 | Weights: [59.0308, -429.2205]
Epoch 86/200 | MSE: 50231877.736334 | Weights: [59.6532, -433.7463]
Epoch 87/200 | MSE: 50223539.970873 | Weights: [60.2741, -438.2609]
Epoch 88/200 | MSE: 50215243.842232 | Weights: [60.8934, -442.7641]
Epoch 89/200 | MSE: 50206989.142482 | Weights: [61.5112, -447.2560]
Epoch 90/200 | MSE: 50198775.664737 | Weights: [62.1274, -451.7368]
Epoch 91/200 | MSE: 50190603.203151 | Weights: [62.7421, -456.2063]
Epoch 92/200 | MSE: 50182471.552893 | Weights: [63.3553, -460.6647]
Epoch 93/200 | MSE: 50174380.510161 | Weights: [63.9669, -465.1119]
Epoch 94/200 | MSE: 50166329.872171 | Weights: [64.5770, -469.5480]
Epoch 95/200 | MSE: 50158319.437155 | Weights: [65.1856, -473.9730]
Epoch 96/200 | MSE: 50150349.004344 | Weights: [65.7926, -478.3869]
Epoch 97/200 | MSE: 50142418.373979 | Weights: [66.3982, -482.7898]
Epoch 98/200 | MSE: 50134527.347296 | Weights: [67.0022, -487.1817]
Epoch 99/200 | MSE: 50126675.726523 | Weights: [67.6047, -491.5626]
Epoch 100/200 | MSE: 50118863.314878 | Weights: [68.2057, -495.9326]
Epoch 101/200 | MSE: 50111089.916556 | Weights: [68.8052, -500.2917]
Epoch 102/200 | MSE: 50103355.336737 | Weights: [69.4032, -504.6398]
Epoch 103/200 | MSE: 50095659.381571 | Weights: [69.9997, -508.9771]
Epoch 104/200 | MSE: 50088001.858175 | Weights: [70.5947, -513.3035]
Epoch 105/200 | MSE: 50080382.574627 | Weights: [71.1882, -517.6191]
Epoch 106/200 | MSE: 50072801.339970 | Weights: [71.7803, -521.9239]
Epoch 107/200 | MSE: 50065257.964195 | Weights: [72.3708, -526.2180]
Epoch 108/200 | MSE: 50057752.258246 | Weights: [72.9599, -530.5013]
Epoch 109/200 | MSE: 50050284.034006 | Weights: [73.5475, -534.7740]
Epoch 110/200 | MSE: 50042853.104301 | Weights: [74.1337, -539.0359]
Epoch 111/200 | MSE: 50035459.282892 | Weights: [74.7184, -543.2872]
Epoch 112/200 | MSE: 50028102.384469 | Weights: [75.3016, -547.5278]
Epoch 113/200 | MSE: 50020782.224647 | Weights: [75.8833, -551.7579]
Epoch 114/200 | MSE: 50013498.619962 | Weights: [76.4637, -555.9774]
Epoch 115/200 | MSE: 50006251.387865 | Weights: [77.0425, -560.1863]
Epoch 116/200 | MSE: 49999040.346721 | Weights: [77.6199, -564.3847]
Epoch 117/200 | MSE: 49991865.315803 | Weights: [78.1959, -568.5726]
Epoch 118/200 | MSE: 49984726.115281 | Weights: [78.7704, -572.7501]
Epoch 119/200 | MSE: 49977622.566228 | Weights: [79.3435, -576.9171]
Epoch 120/200 | MSE: 49970554.490611 | Weights: [79.9151, -581.0736]
```

```
Epoch 121/200 | MSE: 49963521.711284 | Weights: [80.4854, -585.2198]
Epoch 122/200 | MSE: 49956524.051981 | Weights: [81.0542, -589.3556]
Epoch 123/200 | MSE: 49949561.337326 | Weights: [81.6216, -593.4811]
Epoch 124/200 | MSE: 49942633.392811 | Weights: [82.1875, -597.5963]
Epoch 125/200 | MSE: 49935740.044803 | Weights: [82.7521, -601.7012]
Epoch 126/200 | MSE: 49928881.120535 | Weights: [83.3152, -605.7958]
Epoch 127/200 | MSE: 49922056.448103 | Weights: [83.8769, -609.8802]
Epoch 128/200 | MSE: 49915265.856463 | Weights: [84.4372, -613.9544]
Epoch 129/200 | MSE: 49908509.175422 | Weights: [84.9962, -618.0184]
Epoch 130/200 | MSE: 49901786.235640 | Weights: [85.5537, -622.0722]
Epoch 131/200 | MSE: 49895096.868617 | Weights: [86.1098, -626.1159]
Epoch 132/200 | MSE: 49888440.906705 | Weights: [86.6646, -630.1495]
Epoch 133/200 | MSE: 49881818.183082 | Weights: [87.2179, -634.1730]
Epoch 134/200 | MSE: 49875228.531766 | Weights: [87.7699, -638.1864]
Epoch 135/200 | MSE: 49868671.787602 | Weights: [88.3205, -642.1898]
Epoch 136/200 | MSE: 49862147.786256 | Weights: [88.8697, -646.1832]
Epoch 137/200 | MSE: 49855656.364225 | Weights: [89.4175, -650.1666]
Epoch 138/200 | MSE: 49849197.358809 | Weights: [89.9640, -654.1401]
Epoch 139/200 | MSE: 49842770.608130 | Weights: [90.5091, -658.1036]
Epoch 140/200 | MSE: 49836375.951115 | Weights: [91.0528, -662.0572]
Epoch 141/200 | MSE: 49830013.227496 | Weights: [91.5952, -666.0009]
Epoch 142/200 | MSE: 49823682.277807 | Weights: [92.1362, -669.9348]
Epoch 143/200 | MSE: 49817382.943374 | Weights: [92.6759, -673.8588]
Epoch 144/200 | MSE: 49811115.066319 | Weights: [93.2142, -677.7731]
Epoch 145/200 | MSE: 49804878.489551 | Weights: [93.7512, -681.6775]
Epoch 146/200 | MSE: 49798673.056764 | Weights: [94.2869, -685.5722]
Epoch 147/200 | MSE: 49792498.612431 | Weights: [94.8212, -689.4571]
Epoch 148/200 | MSE: 49786355.001806 | Weights: [95.3541, -693.3324]
Epoch 149/200 | MSE: 49780242.070910 | Weights: [95.8858, -697.1979]
Epoch 150/200 | MSE: 49774159.666537 | Weights: [96.4161, -701.0538]
Epoch 151/200 | MSE: 49768107.636245 | Weights: [96.9450, -704.9000]
Epoch 152/200 | MSE: 49762085.828352 | Weights: [97.4727, -708.7366]
Epoch 153/200 | MSE: 49756094.091937 | Weights: [97.9990, -712.5637]
Epoch 154/200 | MSE: 49750132.276828 | Weights: [98.5240, -716.3811]
Epoch 155/200 | MSE: 49744200.233604 | Weights: [99.0477, -720.1891]
Epoch 156/200 | MSE: 49738297.813594 | Weights: [99.5701, -723.9875]
Epoch 157/200 | MSE: 49732424.868866 | Weights: [100.0912, -727.7764]
Epoch 158/200 | MSE: 49726581.252225 | Weights: [100.6110, -731.5558]
Epoch 159/200 | MSE: 49720766.817220 | Weights: [101.1295, -735.3258]
Epoch 160/200 | MSE: 49714981.418116 | Weights: [101.6467, -739.0863]
Epoch 161/200 | MSE: 49709224.909921 | Weights: [102.1626, -742.8375]
Epoch 162/200 | MSE: 49703497.148358 | Weights: [102.6772, -746.5793]
Epoch 163/200 | MSE: 49697797.989874 | Weights: [103.1905, -750.3117]
Epoch 164/200 | MSE: 49692127.291631 | Weights: [103.7025, -754.0348]
Epoch 165/200 | MSE: 49686484.911508 | Weights: [104.2133, -757.7486]
Epoch 166/200 | MSE: 49680870.708088 | Weights: [104.7228, -761.4531]
Epoch 167/200 | MSE: 49675284.540666 | Weights: [105.2310, -765.1483]
Epoch 168/200 | MSE: 49669726.269235 | Weights: [105.7379, -768.8343]
Epoch 169/200 | MSE: 49664195.754487 | Weights: [106.2436, -772.5111]
Epoch 170/200 | MSE: 49658692.857818 | Weights: [106.7480, -776.1787]
Epoch 171/200 | MSE: 49653217.441303 | Weights: [107.2511, -779.8371]
Epoch 172/200 | MSE: 49647769.367717 | Weights: [107.7530, -783.4864]
Epoch 173/200 | MSE: 49642348.500514 | Weights: [108.2537, -787.1265]
Epoch 174/200 | MSE: 49636954.703833 | Weights: [108.7530, -790.7576]
Epoch 175/200 | MSE: 49631587.842487 | Weights: [109.2512, -794.3796]
Epoch 176/200 | MSE: 49626247.781974 | Weights: [109.7481, -797.9925]
Epoch 177/200 | MSE: 49620934.388449 | Weights: [110.2437, -801.5964]
Epoch 178/200 | MSE: 49615647.528751 | Weights: [110.7381, -805.1913]
Epoch 179/200 | MSE: 49610387.070372 | Weights: [111.2313, -808.7772]
Epoch 180/200 | MSE: 49605152.881470 | Weights: [111.7232, -812.3541]
```

```
Epoch 181/200 | MSE: 49599944.830864 | Weights: [112.2139, -815.9221]
Epoch 182/200 | MSE: 49594762.788022 | Weights: [112.7034, -819.4812]
Epoch 183/200 | MSE: 49589606.623070 | Weights: [113.1917, -823.0313]
Epoch 184/200 | MSE: 49584476.206779 | Weights: [113.6787, -826.5726]
Epoch 185/200 | MSE: 49579371.410568 | Weights: [114.1645, -830.1051]
Epoch 186/200 | MSE: 49574292.106494 | Weights: [114.6491, -833.6287]
Epoch 187/200 | MSE: 49569238.167257 | Weights: [115.1325, -837.1435]
Epoch 188/200 | MSE: 49564209.466190 | Weights: [115.6147, -840.6495]
Epoch 189/200 | MSE: 49559205.877259 | Weights: [116.0957, -844.1467]
Epoch 190/200 | MSE: 49554227.275062 | Weights: [116.5754, -847.6352]
Epoch 191/200 | MSE: 49549273.534821 | Weights: [117.0540, -851.1150]
Epoch 192/200 | MSE: 49544344.532379 | Weights: [117.5314, -854.5861]
Epoch 193/200 | MSE: 49539440.144204 | Weights: [118.0076, -858.0485]
Epoch 194/200 | MSE: 49534560.247376 | Weights: [118.4826, -861.5023]
Epoch 195/200 | MSE: 49529704.719591 | Weights: [118.9564, -864.9474]
Epoch 196/200 | MSE: 49524873.439159 | Weights: [119.4290, -868.3839]
Epoch 197/200 | MSE: 49520066.284992 | Weights: [119.9005, -871.8118]
Epoch 198/200 | MSE: 49515283.136608 | Weights: [120.3707, -875.2311]
Epoch 199/200 | MSE: 49510523.874131 | Weights: [120.8398, -878.6419]
Epoch 200/200 | MSE: 49505788.378278 | Weights: [121.3077, -882.0442]
Epoch 1/200 | MSE: 49501076.530366 | Weights: [1.8590, -4.7311]
Epoch 2/200 | MSE: 49490753.910002 | Weights: [3.7133, -9.4503]
Epoch 3/200 | MSE: 49480482.838224 | Weights: [5.5629, -14.1578]
Epoch 4/200 | MSE: 49470263.057611 | Weights: [7.4080, -18.8535]
Epoch 5/200 | MSE: 49460094.312026 | Weights: [9.2484, -23.5374]
Epoch 6/200 | MSE: 49449976.346616 | Weights: [11.0843, -28.2097]
Epoch 7/200 | MSE: 49439908.907796 | Weights: [12.9155, -32.8702]
Epoch 8/200 | MSE: 49429891.743249 | Weights: [14.7422, -37.5191]
Epoch 9/200 | MSE: 49419924.601917 | Weights: [16.5643, -42.1564]
Epoch 10/200 | MSE: 49410007.233995 | Weights: [18.3818, -46.7821]
Epoch 11/200 | MSE: 49400139.390931 | Weights: [20.1949, -51.3962]
Epoch 12/200 | MSE: 49390320.825408 | Weights: [22.0033, -55.9988]
Epoch 13/200 | MSE: 49380551.291348 | Weights: [23.8073, -60.5899]
Epoch 14/200 | MSE: 49370830.543897 | Weights: [25.6067, -65.1695]
Epoch 15/200 | MSE: 49361158.339428 | Weights: [27.4017, -69.7376]
Epoch 16/200 | MSE: 49351534.435531 | Weights: [29.1921, -74.2944]
Epoch 17/200 | MSE: 49341958.591004 | Weights: [30.9781, -78.8397]
Epoch 18/200 | MSE: 49332430.565850 | Weights: [32.7596, -83.3737]
Epoch 19/200 | MSE: 49322950.121272 | Weights: [34.5367, -87.8964]
Epoch 20/200 | MSE: 49313517.019663 | Weights: [36.3093, -92.4077]
Epoch 21/200 | MSE: 49304131.024608 | Weights: [38.0775, -96.9078]
Epoch 22/200 | MSE: 49294791.900864 | Weights: [39.8412, -101.3966]
Epoch 23/200 | MSE: 49285499.414369 | Weights: [41.6006, -105.8742]
Epoch 24/200 | MSE: 49276253.332229 | Weights: [43.3555, -110.3406]
Epoch 25/200 | MSE: 49267053.422713 | Weights: [45.1061, -114.7958]
Epoch 26/200 | MSE: 49257899.455243 | Weights: [46.8523, -119.2399]
Epoch 27/200 | MSE: 49248791.200398 | Weights: [48.5941, -123.6729]
Epoch 28/200 | MSE: 49239728.429902 | Weights: [50.3316, -128.0948]
Epoch 29/200 | MSE: 49230710.916616 | Weights: [52.0647, -132.5056]
Epoch 30/200 | MSE: 49221738.434536 | Weights: [53.7935, -136.9054]
Epoch 31/200 | MSE: 49212810.758790 | Weights: [55.5180, -141.2942]
Epoch 32/200 | MSE: 49203927.665622 | Weights: [57.2382, -145.6721]
Epoch 33/200 | MSE: 49195088.932404 | Weights: [58.9540, -150.0390]
Epoch 34/200 | MSE: 49186294.337606 | Weights: [60.6656, -154.3950]
Epoch 35/200 | MSE: 49177543.660819 | Weights: [62.3729, -158.7400]
Epoch 36/200 | MSE: 49168836.682723 | Weights: [64.0759, -163.0743]
Epoch 37/200 | MSE: 49160173.185099 | Weights: [65.7747, -167.3977]
Epoch 38/200 | MSE: 49151552.950817 | Weights: [67.4692, -171.7103]
Epoch 39/200 | MSE: 49142975.763829 | Weights: [69.1595, -176.0121]
Epoch 40/200 | MSE: 49134441.409169 | Weights: [70.8456, -180.3031]
```

```
Epoch 41/200 | MSE: 49125949.672941 | Weights: [72.5274, -184.5834]
Epoch 42/200 | MSE: 49117500.342323 | Weights: [74.2051, -188.8531]
Epoch 43/200 | MSE: 49109093.205550 | Weights: [75.8785, -193.1120]
Epoch 44/200 | MSE: 49100728.051914 | Weights: [77.5478, -197.3603]
Epoch 45/200 | MSE: 49092404.671765 | Weights: [79.2129, -201.5980]
Epoch 46/200 | MSE: 49084122.856497 | Weights: [80.8738, -205.8251]
Epoch 47/200 | MSE: 49075882.398541 | Weights: [82.5306, -210.0416]
Epoch 48/200 | MSE: 49067683.091374 | Weights: [84.1832, -214.2476]
Epoch 49/200 | MSE: 49059524.729497 | Weights: [85.8317, -218.4430]
Epoch 50/200 | MSE: 49051407.108440 | Weights: [87.4761, -222.6280]
Epoch 51/200 | MSE: 49043330.024754 | Weights: [89.1163, -226.8025]
Epoch 52/200 | MSE: 49035293.276002 | Weights: [90.7525, -230.9666]
Epoch 53/200 | MSE: 49027296.660764 | Weights: [92.3846, -235.1202]
Epoch 54/200 | MSE: 49019339.978626 | Weights: [94.0126, -239.2635]
Epoch 55/200 | MSE: 49011423.030169 | Weights: [95.6365, -243.3964]
Epoch 56/200 | MSE: 49003545.616971 | Weights: [97.2564, -247.5190]
Epoch 57/200 | MSE: 48995707.541607 | Weights: [98.8722, -251.6313]
Epoch 58/200 | MSE: 48987908.607632 | Weights: [100.4840, -255.7333]
Epoch 59/200 | MSE: 48980148.619583 | Weights: [102.0917, -259.8251]
Epoch 60/200 | MSE: 48972427.382974 | Weights: [103.6955, -263.9066]
Epoch 61/200 | MSE: 48964744.704291 | Weights: [105.2952, -267.9779]
Epoch 62/200 | MSE: 48957100.390983 | Weights: [106.8909, -272.0390]
Epoch 63/200 | MSE: 48949494.251466 | Weights: [108.4826, -276.0900]
Epoch 64/200 | MSE: 48941926.095109 | Weights: [110.0704, -280.1309]
Epoch 65/200 | MSE: 48934395.732232 | Weights: [111.6542, -284.1616]
Epoch 66/200 | MSE: 48926902.974104 | Weights: [113.2340, -288.1823]
Epoch 67/200 | MSE: 48919447.632936 | Weights: [114.8099, -292.1929]
Epoch 68/200 | MSE: 48912029.521881 | Weights: [116.3818, -296.1935]
Epoch 69/200 | MSE: 48904648.455017 | Weights: [117.9498, -300.1841]
Epoch 70/200 | MSE: 48897304.247354 | Weights: [119.5139, -304.1647]
Epoch 71/200 | MSE: 48889996.714830 | Weights: [121.0741, -308.1354]
Epoch 72/200 | MSE: 48882725.674296 | Weights: [122.6303, -312.0962]
Epoch 73/200 | MSE: 48875490.943519 | Weights: [124.1827, -316.0470]
Epoch 74/200 | MSE: 48868292.341180 | Weights: [125.7312, -319.9880]
Epoch 75/200 | MSE: 48861129.686863 | Weights: [127.2759, -323.9191]
Epoch 76/200 | MSE: 48854002.801049 | Weights: [128.8166, -327.8403]
Epoch 77/200 | MSE: 48846911.505123 | Weights: [130.3535, -331.7518]
Epoch 78/200 | MSE: 48839855.621354 | Weights: [131.8866, -335.6535]
Epoch 79/200 | MSE: 48832834.972904 | Weights: [133.4159, -339.5455]
Epoch 80/200 | MSE: 48825849.383819 | Weights: [134.9413, -343.4277]
Epoch 81/200 | MSE: 48818898.679020 | Weights: [136.4629, -347.3002]
Epoch 82/200 | MSE: 48811982.684300 | Weights: [137.9807, -351.1630]
Epoch 83/200 | MSE: 48805101.226331 | Weights: [139.4947, -355.0162]
Epoch 84/200 | MSE: 48798254.132644 | Weights: [141.0049, -358.8597]
Epoch 85/200 | MSE: 48791441.231629 | Weights: [142.5114, -362.6937]
Epoch 86/200 | MSE: 48784662.352541 | Weights: [144.0140, -366.5180]
Epoch 87/200 | MSE: 48777917.325476 | Weights: [145.5130, -370.3328]
Epoch 88/200 | MSE: 48771205.981392 | Weights: [147.0081, -374.1381]
Epoch 89/200 | MSE: 48764528.152082 | Weights: [148.4996, -377.9338]
Epoch 90/200 | MSE: 48757883.670184 | Weights: [149.9873, -381.7200]
Epoch 91/200 | MSE: 48751272.369165 | Weights: [151.4713, -385.4968]
Epoch 92/200 | MSE: 48744694.083333 | Weights: [152.9516, -389.2642]
Epoch 93/200 | MSE: 48738148.647816 | Weights: [154.4281, -393.0221]
Epoch 94/200 | MSE: 48731635.898566 | Weights: [155.9010, -396.7706]
Epoch 95/200 | MSE: 48725155.672357 | Weights: [157.3702, -400.5098]
Epoch 96/200 | MSE: 48718707.806780 | Weights: [158.8358, -404.2396]
Epoch 97/200 | MSE: 48712292.140230 | Weights: [160.2976, -407.9600]
Epoch 98/200 | MSE: 48705908.511914 | Weights: [161.7559, -411.6712]
Epoch 99/200 | MSE: 48699556.761844 | Weights: [163.2104, -415.3731]
Epoch 100/200 | MSE: 48693236.730823 | Weights: [164.6614, -419.0658]
```

```
Epoch 101/200 | MSE: 48686948.260461 | Weights: [166.1087, -422.7492]
Epoch 102/200 | MSE: 48680691.193145 | Weights: [167.5524, -426.4234]
Epoch 103/200 | MSE: 48674465.372059 | Weights: [168.9924, -430.0884]
Epoch 104/200 | MSE: 48668270.641168 | Weights: [170.4289, -433.7443]
Epoch 105/200 | MSE: 48662106.845215 | Weights: [171.8618, -437.3910]
Epoch 106/200 | MSE: 48655973.829715 | Weights: [173.2911, -441.0286]
Epoch 107/200 | MSE: 48649871.440964 | Weights: [174.7168, -444.6571]
Epoch 108/200 | MSE: 48643799.526016 | Weights: [176.1390, -448.2765]
Epoch 109/200 | MSE: 48637757.932693 | Weights: [177.5576, -451.8869]
Epoch 110/200 | MSE: 48631746.509577 | Weights: [178.9727, -455.4883]
Epoch 111/200 | MSE: 48625765.106006 | Weights: [180.3842, -459.0807]
Epoch 112/200 | MSE: 48619813.572066 | Weights: [181.7922, -462.6640]
Epoch 113/200 | MSE: 48613891.758604 | Weights: [183.1967, -466.2385]
Epoch 114/200 | MSE: 48607999.517193 | Weights: [184.5976, -469.8039]
Epoch 115/200 | MSE: 48602136.700167 | Weights: [185.9951, -473.3605]
Epoch 116/200 | MSE: 48596303.160579 | Weights: [187.3891, -476.9082]
Epoch 117/200 | MSE: 48590498.752231 | Weights: [188.7796, -480.4470]
Epoch 118/200 | MSE: 48584723.329647 | Weights: [190.1666, -483.9770]
Epoch 119/200 | MSE: 48578976.748081 | Weights: [191.5501, -487.4981]
Epoch 120/200 | MSE: 48573258.863505 | Weights: [192.9302, -491.0104]
Epoch 121/200 | MSE: 48567569.532616 | Weights: [194.3068, -494.5140]
Epoch 122/200 | MSE: 48561908.612822 | Weights: [195.6800, -498.0088]
Epoch 123/200 | MSE: 48556275.962248 | Weights: [197.0498, -501.4948]
Epoch 124/200 | MSE: 48550671.439723 | Weights: [198.4161, -504.9722]
Epoch 125/200 | MSE: 48545094.904781 | Weights: [199.7790, -508.4408]
Epoch 126/200 | MSE: 48539546.217660 | Weights: [201.1386, -511.9008]
Epoch 127/200 | MSE: 48534025.239296 | Weights: [202.4947, -515.3521]
Epoch 128/200 | MSE: 48528531.831316 | Weights: [203.8474, -518.7948]
Epoch 129/200 | MSE: 48523065.856046 | Weights: [205.1967, -522.2289]
Epoch 130/200 | MSE: 48517627.176487 | Weights: [206.5427, -525.6544]
Epoch 131/200 | MSE: 48512215.656334 | Weights: [207.8853, -529.0714]
Epoch 132/200 | MSE: 48506831.159962 | Weights: [209.2245, -532.4798]
Epoch 133/200 | MSE: 48501473.552417 | Weights: [210.5604, -535.8797]
Epoch 134/200 | MSE: 48496142.699426 | Weights: [211.8930, -539.2710]
Epoch 135/200 | MSE: 48490838.467380 | Weights: [213.2222, -542.6539]
Epoch 136/200 | MSE: 48485560.723344 | Weights: [214.5481, -546.0284]
Epoch 137/200 | MSE: 48480309.335042 | Weights: [215.8707, -549.3944]
Epoch 138/200 | MSE: 48475084.170861 | Weights: [217.1900, -552.7520]
Epoch 139/200 | MSE: 48469885.099843 | Weights: [218.5060, -556.1012]
Epoch 140/200 | MSE: 48464711.991686 | Weights: [219.8187, -559.4420]
Epoch 141/200 | MSE: 48459564.716739 | Weights: [221.1281, -562.7745]
Epoch 142/200 | MSE: 48454443.145994 | Weights: [222.4342, -566.0986]
Epoch 143/200 | MSE: 48449347.151093 | Weights: [223.7371, -569.4145]
Epoch 144/200 | MSE: 48444276.604317 | Weights: [225.0367, -572.7220]
Epoch 145/200 | MSE: 48439231.378587 | Weights: [226.3331, -576.0213]
Epoch 146/200 | MSE: 48434211.347449 | Weights: [227.6262, -579.3123]
Epoch 147/200 | MSE: 48429216.385093 | Weights: [228.9161, -582.5951]
Epoch 148/200 | MSE: 48424246.366330 | Weights: [230.2028, -585.8697]
Epoch 149/200 | MSE: 48419301.166600 | Weights: [231.4862, -589.1361]
Epoch 150/200 | MSE: 48414380.661958 | Weights: [232.7665, -592.3944]
Epoch 151/200 | MSE: 48409484.729089 | Weights: [234.0435, -595.6445]
Epoch 152/200 | MSE: 48404613.245283 | Weights: [235.3174, -598.8864]
Epoch 153/200 | MSE: 48399766.088449 | Weights: [236.5880, -602.1203]
Epoch 154/200 | MSE: 48394943.137107 | Weights: [237.8555, -605.3461]
Epoch 155/200 | MSE: 48390144.270376 | Weights: [239.1198, -608.5638]
Epoch 156/200 | MSE: 48385369.367986 | Weights: [240.3810, -611.7735]
Epoch 157/200 | MSE: 48380618.310266 | Weights: [241.6390, -614.9751]
Epoch 158/200 | MSE: 48375890.978139 | Weights: [242.8939, -618.1687]
Epoch 159/200 | MSE: 48371187.253127 | Weights: [244.1456, -621.3544]
Epoch 160/200 | MSE: 48366507.017342 | Weights: [245.3942, -624.5321]
```

```
Epoch 161/200 | MSE: 48361850.153485 | Weights: [246.6397, -627.7018]
Epoch 162/200 | MSE: 48357216.544841 | Weights: [247.8820, -630.8637]
Epoch 163/200 | MSE: 48352606.075280 | Weights: [249.1213, -634.0176]
Epoch 164/200 | MSE: 48348018.629253 | Weights: [250.3574, -637.1636]
Epoch 165/200 | MSE: 48343454.091782 | Weights: [251.5905, -640.3018]
Epoch 166/200 | MSE: 48338912.348473 | Weights: [252.8205, -643.4321]
Epoch 167/200 | MSE: 48334393.285492 | Weights: [254.0474, -646.5546]
Epoch 168/200 | MSE: 48329896.789584 | Weights: [255.2712, -649.6693]
Epoch 169/200 | MSE: 48325422.748051 | Weights: [256.4920, -652.7762]
Epoch 170/200 | MSE: 48320971.048764 | Weights: [257.7097, -655.8754]
Epoch 171/200 | MSE: 48316541.580149 | Weights: [258.9244, -658.9668]
Epoch 172/200 | MSE: 48312134.231194 | Weights: [260.1361, -662.0504]
Epoch 173/200 | MSE: 48307748.891437 | Weights: [261.3447, -665.1264]
Epoch 174/200 | MSE: 48303385.450970 | Weights: [262.5503, -668.1946]
Epoch 175/200 | MSE: 48299043.800436 | Weights: [263.7529, -671.2552]
Epoch 176/200 | MSE: 48294723.831017 | Weights: [264.9525, -674.3082]
Epoch 177/200 | MSE: 48290425.434447 | Weights: [266.1490, -677.3535]
Epoch 178/200 | MSE: 48286148.502993 | Weights: [267.3426, -680.3912]
Epoch 179/200 | MSE: 48281892.929466 | Weights: [268.5332, -683.4213]
Epoch 180/200 | MSE: 48277658.607210 | Weights: [269.7208, -686.4438]
Epoch 181/200 | MSE: 48273445.430102 | Weights: [270.9055, -689.4588]
Epoch 182/200 | MSE: 48269253.292546 | Weights: [272.0872, -692.4662]
Epoch 183/200 | MSE: 48265082.089476 | Weights: [273.2659, -695.4661]
Epoch 184/200 | MSE: 48260931.716353 | Weights: [274.4417, -698.4585]
Epoch 185/200 | MSE: 48256802.069153 | Weights: [275.6146, -701.4435]
Epoch 186/200 | MSE: 48252693.044381 | Weights: [276.7845, -704.4210]
Epoch 187/200 | MSE: 48248604.539052 | Weights: [277.9515, -707.3910]
Epoch 188/200 | MSE: 48244536.450694 | Weights: [279.1156, -710.3536]
Epoch 189/200 | MSE: 48240488.677355 | Weights: [280.2768, -713.3088]
Epoch 190/200 | MSE: 48236461.117582 | Weights: [281.4350, -716.2566]
Epoch 191/200 | MSE: 48232453.670437 | Weights: [282.5904, -719.1970]
Epoch 192/200 | MSE: 48228466.235481 | Weights: [283.7429, -722.1301]
Epoch 193/200 | MSE: 48224498.712778 | Weights: [284.8925, -725.0559]
Epoch 194/200 | MSE: 48220551.002891 | Weights: [286.0392, -727.9743]
Epoch 195/200 | MSE: 48216623.006880 | Weights: [287.1831, -730.8855]
Epoch 196/200 | MSE: 48212714.626300 | Weights: [288.3241, -733.7893]
Epoch 197/200 | MSE: 48208825.763195 | Weights: [289.4622, -736.6859]
Epoch 198/200 | MSE: 48204956.320102 | Weights: [290.5975, -739.5753]
Epoch 199/200 | MSE: 48201106.200037 | Weights: [291.7300, -742.4574]
Epoch 200/200 | MSE: 48197275.306511 | Weights: [292.8596, -745.3324]
  2Ù§ü   - FAILED!sing lms_adaptive_filter :  [ÍÚ  Äã
```